

µCSS

A Node-based CSS and web-graphics processor.

npm package: gulp-mu-css

Manual

Version 2.5.18

2026-09-24

Contents

1 Introduction....5
1.1 Package versions and µGulp ready....5
1.2 Tooling and cost....6
1.3 Draft workflow (authoring + watch)....6
1.3.1 Photopea (browser, optional)....7
1.4 Context: Why µCSS™?....7
2 Installation and getting started....9
3 Use cases....10
3.1 Named properties (variables)....10
3.2 Calculations and expressions....10
3.3 Multiple layouts via skin manifests....11
3.4 Automatic sprite generation....11
3.5 Inlining images (DataUri)....12
3.6 Preloading images (preload)....13
3.7 Cursors with hotspot and fallback....13
3.8 Automatic image generation (media steps)....13
3.8.1 Custom media steps (plugins)....14
4 microPS (µPS) — image generators....17
4.1 API overview....17
4.2 ButtonAndIconCreator....17
4.2.1 CreateByTopLayerSets — one image per group....18
4.3 AppIconMaker....18
4.4 SpriteAtlas and TileSheet....19
4.5 SequenceStrip and DSD format....19
4.5.1 Authoring DSD (`CreateDsdFromImages`)....20
4.5.2 PSD sequence groups....20
4.6 Raster model, I/O and canvas size....21
4.7 Adjustments and masks....21
4.7.1 Masks....21
4.7.2 Selection engine....22
4.7.3 ApplyGamma....22
4.7.4 ApplyBrightnessContrast....22
4.7.5 ApplyHueSaturation....22
4.7.6 ApplyAdjustmentStack....22
4.8 Raster transforms....23
4.9 ExtendScript entry point (`MuPsDoc`)....23
4.10 PSD compositor and layer compositing....24
4.10.1 Reproduced layer effects (layer styles)....24
4.10.2 Style transfer (ButtonAndIconCreator) vs. layer links....26
4.10.3 Opacity model (three independent controls)....26
4.10.4 Reference PSD and Adobe ground truth....27
4.10.5 Tools (µPS development)....27
4.11 Draft workflow (authoring + watch)....27
4.11.1 Intermediate steps raw → final (`outputBase`)....28
5 microAU (µAU) — sound atlas....29
5.1 API overview....29
5.2 SoundAtlasMaker....29
5.2.1 Loop points in file names....29
5.2.2 JSON format....30

5.2.3 MP3 inputs....30
5.3 µCSS™ integration (manifest `sounds`)....30
5.3.1 Planned: CSS sounds, auto-wiring & handler overrides....30
6 microFT (µFT) — icon font....31
6.1 Glyph naming....31
6.2 FontGenerator....31
6.3 API building blocks....32
6.4 Color fonts and emoji fonts....32
6.5 µCSS™ integration....32
7 Vue and component co-location....33
7.1 Workflow....33
7.2 Namespaces and global state classes....33
7.3 Migrating existing Vue SFCs....33
8 Core ideas of µCSS™....34
8.1 The .µ.css format....34
8.2 Value interpolation µ(expression)....34
8.3 Directives -µ: expression....34
8.4 The skin manifest....35
8.5 JavaScript without substitute characters....35
9 Build process and Gulp integration....36
9.1 Directory conventions....36
9.2 Compilation flow....36
9.3 Incremental build and cache....36
9.4 Gulp tasks and watch....37
10 Manifest reference (DefineSkin)....38
10.1 vars....38
10.2 helpers....38
10.3 cursors....38
10.4 media....38
10.5 imageFormat....39
10.6 sprites....40
10.7 minify....41
10.8 sounds....41
10.9 font....42
10.10 files....43
11 µ evaluation context (reference)....44
11.1 The $ object....44
11.2 Color and value functions....44
11.3 Rule and document methods....44
11.4 Sprite()....45
11.5 Cursor()....45
11.6 DataUri()....45
11.7 Helper functions and the this binding....46
11.8 afterWork hooks....46
12 Working with colors....47
12.1 Defining colors....47
12.2 Alpha values....47
12.3 Color computations....47
13 Node API....48
13.1 Version history API (RELEASES.json)....49
14 Error diagnostics....50

15 Migrating from µCSS™.....51
16 Version history.....52
17 Legal.....55
17.1 MIT license (original text).....55
17.2 Trademarks.....55
17.3 Third-party libraries.....55

1 Introduction



µCSS™ logo

µCSS™ is a Node module for processing CSS files and generating web graphics. This manual describes version 2 — the Node-based successor of µCSS™ 1, introduced in 2013 as an Adobe Photoshop script: from enhanced source stylesheets (`.µ.css`) and the media sources (e.g. PSD drafts), Gulp produces the finished skin files — standard CSS plus all required images, sprites, cursors, fonts and sounds. Image generation is handled by the sibling module µPS; sound atlases and icon fonts by µAU and µFT — each in its own chapter (*microPS*, *microAU*, *microFT*).

Because the µ character keeps causing problems in package and repository names (npm, git), the technical names are `gulp-mu-css` and `gulp-mu-ps` — µCSS™ and µPS are the display names.

The key features of µCSS™ 2:

- Source stylesheets stay **syntactically valid CSS** — editors, linters and diff tools work unchanged.
- **Named CSS properties** via skin variables (`$.name`).
- **Computing CSS values with JavaScript expressions** directly in the stylesheet — without substitute characters like `«»¡`.
- **Multiple layouts (skins)** from the same sources via manifest files.
- **Automatic sprite atlas generation** including retina variants (`@2x`) and `image-set()`.
- **Cursor management** with hotspot, fallback and retina support.
- **Preloading of CSS images** via a generated preload rule.
- **Inlining images as data URIs** via `DataUri()` — small assets embedded directly in the CSS (not packed into the sprite atlas).
- **Automatic image generation** from PSD drafts (buttons, icons, app icons, animation strips) via µPS.
- **Incremental build with cache** — only what changed is regenerated.
- **Meaningful error messages** with file, line and source excerpt.

Unlike µCSS™ 1, version 2 runs entirely in Node.js (version 18 and up) and needs neither Photoshop nor any other Adobe product. The build is driven via Gulp or directly through the Node API.

1.1 Package versions and µGulp ready

This edition covers µCSS 2.5.18, µPS 1.3.6, µFT 0.2.0 and µAU 0.1.8.

The four packages are µGulp ready. Public product information and installation links are available at https://microgulp.dev. The task dashboard is optional; all build tasks also work from the ordinary Gulp command line.



µGulp ready

Project pages and npm READMEs link the badge to https://microgulp.dev and load the image from https://microgulp.dev/assets/media/microgulp-ready.png. This public asset does not require access to the private source repository. The manual includes its own copy for offline reading.

This release also includes more robust media-cache handling and animation frame processing. Invalid cache entries trigger regeneration; sequence folders ignore non-image files, and µPS handles unaligned pixel buffers. Media build errors retain their original stack for diagnosis.

1.2 Tooling and cost

Version 2 needs **no Adobe subscription** for the build: sprites, cursors and PSD rendering run headless via Node and µPS.

If you edit layered PSD sources yourself, pick any editor — the build is identical. **Affinity** and **Photopea** are common no-/low-cost options; **Adobe Photoshop** works too if `.psd` is associated with it or you pass the executable path. Save as PSD; µPS reads the same layer structure. With µCSS™ 1, compilation and bitmap generation both depended on a licensed desktop imaging app — recurring cost that drops away for many teams.

1.3 Draft workflow (authoring + watch)

No editor is required for the build. For **opening** drafts from µPS, set `MU_DRAFT_APP` (or `OpenDrafts(…, { app })`):

`app` / env	Editor
`default` (default)	OS default for `.psd` (often Photoshop if installed)
`affinity`	Affinity Photo (`MU_AFFINITY_EXE` optional)
`<path to .exe/.app>`	Explicit app, e.g. Photoshop
`photopea`	Browser — use `await OpenPhotopeaDrafts(…)` (async), not `OpenDrafts`

Automation splits cleanly (same for all editors):

Step	Tool	Role
Authoring	Your editor (Affinity, Photoshop, Photopea, …)	Maintain layered PSD drafts
Trigger	Save PSD → `WatchDrafts` (µPS) or `gulp.watch`	Debounced rebuild (~1.5 s)
Processing	µPS in Node	Layer read, style transfer, compositing, PNG export
Optional	`OpenDrafts` /	Open a draft in the chosen

Step	Tool	Role
	`OpenPhotopeaDrafts`	editor

µPS performs the layer separation and rendering that the old Photoshop script handled; the editor only supplies the source file. See `gulp-mu-ps` API: `OpenDrafts`, `OpenPhotopeaDrafts`, `WatchDrafts`; CLI: `tools/open-drafts.mjs`, `tools/watch-drafts.mjs`. Details: `docs/CONCEPT.md` (D11/D11b).

1.3.1 Photopea (browser, optional)

If you prefer not to install a desktop app, [Photopea](https://www.photopea.com/) is a free online PSD editor (full PSD, local processing). Use **`OpenPhotopeaDrafts`** only when you want the browser — otherwise `OpenDrafts` with Affinity or Photoshop is fine.

```
import { OpenPhotopeaDrafts, WatchDrafts } from "gulp-mu-ps";

WatchDrafts(["dev/media/final/general/gui/buttons.psd"], () => buildSkin());

await OpenPhotopeaDrafts(["dev/media/final/general/gui/buttons.psd"], {
    saveBack: true,
    onSave: ({ path }) => console.log("Updated:", path)
});
```

CLI: `node gulp-mu-ps/tools/open-drafts.mjs --app photopea path/to/draft.psd` (keeps running for save-back until Ctrl+C). Env: `MU_PHOTOPEA_SCRIPT` (optional script after load).

1.4 Context: Why µCSS™?

For almost every individual aspect of µCSS™ an established single-purpose tool exists — but no system that bundles everything:

Feature	Closest existing equivalent	What is missing there
Variables & color functions	Sass/LESS (`lighten()`, `mix()`), native CSS (`color-mix()`, custom properties)	no embedded JavaScript
JavaScript in CSS values	postcss-functions (registered JS functions as CSS functions)	only named functions — no free expressions, no directives with rule/document access
Stylesheets with full JS power	vanilla-extract (stylesheets-in-TypeScript)	the inverse approach — the regular CSS format is lost
Sprite atlas from CSS references	spritesmith family, postcss-sprites	largely frozen, no retina `image-set` workflow, no shared cache
Image generation from PSD drafts	only building blocks (`ag-psd`, asset export from Figma/Sketch)	no series rendering with layer-style transfer, not coupled to the CSS build
Cursors, preload, skin manifest, media cache	hand-written build scripts	bundled nowhere

Part of the original 2013 µCSS™ motivation is solved today by native CSS (custom properties, `color-mix()`, nesting) — which is why µCSS™ 2 deliberately drops LESS support and vendor prefixes. The remaining core is without competition: arbitrary JavaScript expressions in CSS plus directives with AST access, combined with a sprite atlas including retina, a PSD render pipeline and an incremental cache,

driven by one manifest per skin. And since µCSS™ is internally a PostCSS pipeline, the entire PostCSS ecosystem (cssnano, Stylelint, …) stays attachable instead of competing with it.

2 Installation and getting started

µCSS™ and µPS are available as the npm modules `gulp-mu-css` and `gulp-mu-ps`. Install them in your own project:

```
npm install gulp-mu-css gulp-mu-ps
```

µCSS™ depends on µPS (atlas and image generation) — not the other way around. Both modules can be used separately.

The typical entry point is a Gulp task that builds a skin manifest:

```
// gulpfile.mjs
import gulp from "gulp";
import { BuildSkin } from "gulp-mu-css";

export async function SkinStd() {
    await BuildSkin("skins/src/std.µcss.mjs");
}
export function SkinWatch() {
    gulp.watch(["skins/src/**", "dev/media/final/**"], SkinStd);
}
```

Running `npx gulp SkinStd` compiles the skin "std" into the directory `skins/std/`. `BuildSkin` is idempotent and cache-backed: a second run without changes finishes in a few milliseconds (see the chapter "Build process").

3 Use cases

The following sections show the typical usage scenarios — analogous to the use cases of the old µCSS™ manual, but in the new syntax.

3.1 Named properties (variables)

The simplest use case: a value is named once in the skin manifest and used in any number of places. In the manifest:

```
// skins/src/std.µcss.mjs
import { DefineSkin } from "gulp-mu-css";

export default DefineSkin({
    vars: {
        textColor: "#ff0000",
        backColor: "#0000ff"
    },
    files: [{ source: "src.µ.css", target: "std.css" }]
});
```

In the source stylesheet `src.µ.css`:

```
div.mydiv {
    padding: 5px;
    font-size: 24px;
    color: µ($.textColor);
    background-color: µ($.backColor);
    border: 10px µ($.backColor) solid;
    border-radius: 10px;
}
```

Compiled to `std.css`:

```
div.mydiv {
    padding: 5px;
    font-size: 24px;
    color: #ff0000;
    background-color: #0000ff;
    border: 10px #0000ff solid;
    border-radius: 10px;
}
```

Unlike the old µCSS™, no placeholder properties and no `Set*` directives are needed any more: the value sits exactly where it belongs.

3.2 Calculations and expressions

The content of `µ(...)` is an arbitrary JavaScript expression. This allows calculations, conditions and color operations directly in the stylesheet:

```
td.subheadline {
    border-bottom: 1px dashed µ(Lighten($.selectBaseBrgdColor, 0.7));
}
div.panel {
    padding: µ(Math.floor($.basePadding * 0.2))px;
    background-color: µ(Alpha($.baseBrgdColor, 0.85));
    z-index: µ($.PanelZIndex + 10);
}
div.hint {
    color: µ($.darkMode ? "#e0e0e0" : "#202020");
}
```

For operations that produce several properties or change the rule as a whole, there are directives (`-µ:`), for example using your own macro functions from the manifest:

```
div.menu {
    -µ: Borders($.menuBaseBrgdColor, 1, 0.3, -0.3, -0.3, 0.3);
}
```

The directive calls the helper function `Borders`, which computes four `border-*` properties and inserts them into the rule (see the chapter "µ evaluation context").

3.3 Multiple layouts via skin manifests

The old template compiling (one template CSS plus a `µ.layout.css` per layout) is replaced by several manifests that share the same `.µ.css` sources:

```
// skins/src/layout_a.mjs
import { DefineSkin } from "gulp-mu-css";
export default DefineSkin({
    vars: { textColor: "#ff0000", backColor: "#0000ff" },
    files: [{ source: "template.µ.css", target: "layout_a.css" }]
});
```

```
// skins/src/layout_b.mjs
import { DefineSkin } from "gulp-mu-css";
export default DefineSkin({
    vars: { textColor: "#ff00ff", backColor: "#00ff00" },
    files: [{ source: "template.µ.css", target: "layout_b.css" }]
});
```

Each manifest produces its own output directory (`skins/layout_a/`, `skins/layout_b/`) with its own CSS, its own images and its own build cache. Shared variables can be imported as a regular JavaScript module and used in both manifests.

3.4 Automatic sprite generation

One of the highlights of µCSS™ (in version 1 as in version 2) is the automatic generation of sprite atlases. The directive `Sprite(url)` registers a rule's image for the atlas:

```
div.loginbutton {
    display: inline-block;
    -µ: Sprite("imgs/aqua/but_login_normal.png");
}
div.loginbutton:hover {
    display: inline-block;
    -µ: Sprite("imgs/aqua/but_login_hover.png");
}
div.helpbutton {
    display: inline-block;
    -µ: Sprite("imgs/aqua/but_help_normal.png");
}
```

During the build, all registered images are packed into a single atlas image (`imgs/sprites.png`, plus `imgs/sprites@2x.png` from the `@2x` source images) and the rules are rewritten:

```
div.loginbutton {
    display: inline-block;
    background-image: url(imgs/sprites.png);
    background-image: image-set(url(imgs/sprites.png)1x, url(imgs/sprites@2x.png)2x);
    background-repeat: no-repeat;
    background-position: 0px 0px;
    width: 55px;
    height: 55px;
}
```

`width`, `height`, `background-position` and `background-repeat` are set automatically; existing declarations of these properties in the rule are replaced. Unlike the old µCSS™, the vendor-prefixed variants (`-webkit-image-set` etc.) are gone — all relevant browsers have supported `image-set()` unprefixed for years.

Identical source images automatically share one atlas position (deduplication). The atlas is only repacked when the set of images or a source image has changed.

3.5 Inlining images (DataUri)

For **small, self-contained images** (tiny icons, 1×1 placeholders, UI chrome) you can embed the file bytes directly in the compiled CSS as a `data:` URI instead of referencing a separate file or the sprite atlas. **`DataUri()` does not register the image for the atlas** — that would defeat the purpose.

Typical uses: carbon/placeholder textures, mail-template icons, assets that must ship in a single CSS file without extra HTTP requests. For larger sets of similar tiles (buttons, icons), **`Sprite()` remains the better choice**.

Directive (sets `background-image`, replaces any existing `background-image` on the rule):

```
div.carbon {
    -µ: DataUri("imgs/general/gui/carbon.png");
}
```

Value form (for any property):

```
div.badge {
    background-image: µ(DataUri("imgs/general/gui/carbon.png"));
}
```

Retina (requires `@2x` source next to the 1x file, like sprites/cursors):

```
-µ: DataUri("imgs/icon.png", { retina: true });
/* → background-image: image-set(url(data:…)1x, url(data:…)2x); */
```

Option	Default	Description
`url`	— (required)	Image path relative to the skin directory (same resolution as `Sprite()` sources: literal path, then `.png`/`.webp`/…).
`retina`	`false`	When `true`, embeds 1x and `@2x` as an unprefixed `image-set(...)`.
`wrap`	`true`	When `true`, returns `url(data:…)`; when `false`, returns only the `data:…; base64,…` string (e.g. for custom `url()` assembly).
`mime`	(from extension)	Override MIME type (`image/png`, `image/svg+xml`, …).
`property`	`"background-image"`	Directive only: CSS property to write (default replaces `background-image`).

Supported formats: PNG, WebP, JPEG, GIF, SVG. During `BuildSkin`, paths are resolved against the skin output directory (`skins/<skin>/`) — run **media steps first** so generated images exist before compilation.

Migration from legacy base64 manifest vars: instead of `url("+$.CarbonImageURI+")` in a directive, reference the PNG in the skin and use `-µ: DataUri("imgs/general/gui/carbon.png");`.

3.6 Preloading images (preload)

Important images (e.g. hover states and cursors) can be loaded ahead of time when the page loads. µCSS™ collects the image URLs and generates a preload rule when the manifest option `sprites.preloadRule` is set:

```
div.csspreload {
    background-image: url(imgs/sprites.png), url(imgs/general/gui/cursors/zoom.png);
    display: none;
}
```

In the HTML page, an empty element with this class is enough:

```
<div class="csspreload"></div>
```

Cursor images from the `cursors` definitions are added to the preload list automatically.

3.7 Cursors with hotspot and fallback

Cursors are defined in the manifest and used by name in the stylesheet. Definition:

```
cursors: [
    { name: "zoom", fallback: "zoom-in", image: "imgs/general/gui/cursors/zoom.png",
hotspot: [10, 8] },
    { name: "wait", fallback: "wait", image: "imgs/general/gui/cursors/wait.png", hotspot:
[12, 12] }
]
```

Usage as a directive or as a value form:

```
*.cursor_zoom {
    -µ: Cursor("zoom");
}
a.preview {
    cursor: µ(Cursor("zoom"));
}
```

The directive produces the `url()` form with hotspot and fallback and — when an `@2x` image file exists — additionally the `image-set()` variant:

```
*.cursor_zoom {
    cursor: url(imgs/general/gui/cursors/zoom.png) 10 8, zoom-in;
    cursor: image-set(url(imgs/general/gui/cursors/zoom.png)1x,
url(imgs/general/gui/cursors/zoom@2x.png)2x) 10 8, zoom-in;
}
```

3.8 Automatic image generation (media steps)

What the µCSS™ plugins used to do (ButtonCreator, AppIconMaker, FileCopy) is now handled by the `media` entries in the manifest. They run before CSS compilation and generate or copy all media into the skin directory:

```
media: [
    { buttonsAndIcons: "dev/media/final/general/gui/panelbuttons.psd", layout: "std",
outputDir: "imgs/general/gui/panelbuttons" },
```

```
    { buttonsAndIcons: "dev/media/final/general/gui/cursors.psd", layout: "std", outputDir:
"imgs/general/gui/cursors" },
    { appIcons: "dev/media/final/favicon.psd", layout: "std", profiles: ["web"] },
    { sequenceStrip: "dev/media/raw/glittery/imgs", outputFile:
"imgs/general/gui/glittery/glittery.png" },
    { copy: "dev/media/final/general/gui/teaserbgrd.png", to: "imgs/general/gui" },
    { copyFolder: "dev/media/final/fonts", to: "fonts" }
]
```

3.8.1 Custom media steps (plugins)

Built-in steps cover the legacy µCSS™ plugins. For project-specific generators (similar to a custom ButtonAndIconCreator), register a handler **before** `BuildSkin`:

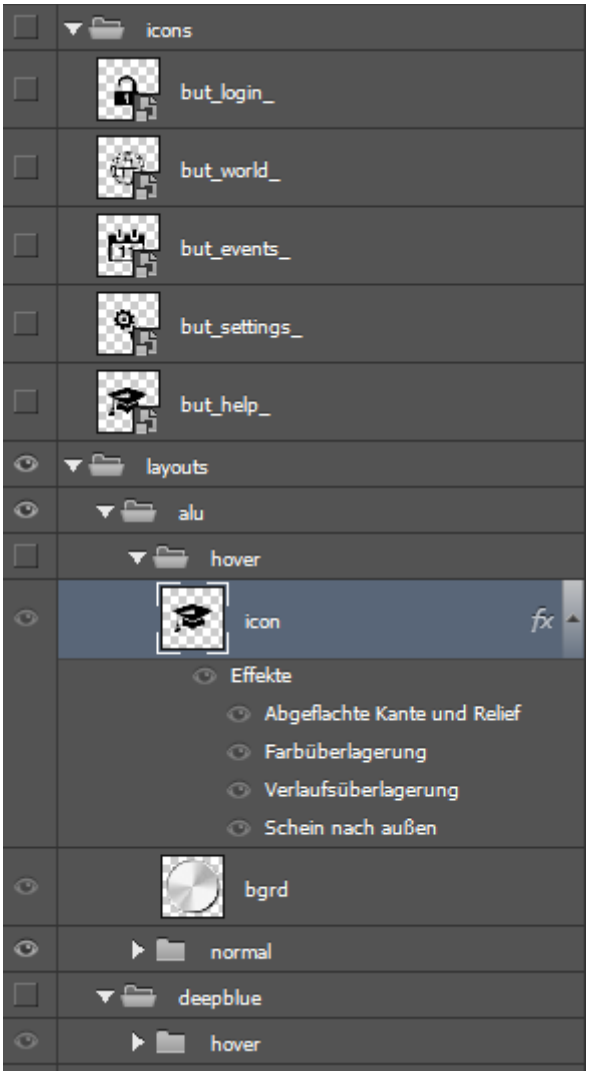
```
import { RegisterMediaStep, BuildSkin } from "gulp-mu-css";

RegisterMediaStep("myTiles", async (step, ctx) => {
    // use gulp-mu-ps primitives or your own logic
    return { type: "myTiles", skipped: false, outputs: [/* absolute paths */] };
});

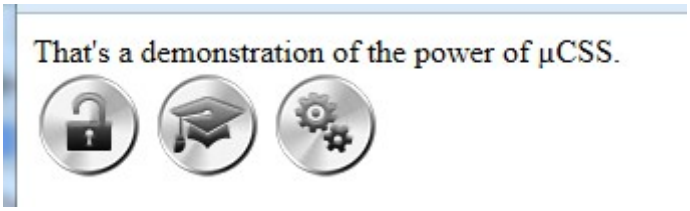
await BuildSkin("mySkin.µcss.mjs");
// manifest: { myTiles: { source: "…", outputDir: "…" } }
```

See `docs/CONCEPT.md` (D13). Built-in keys (`buttonsAndIcons`, `copy`, …) cannot be overridden.

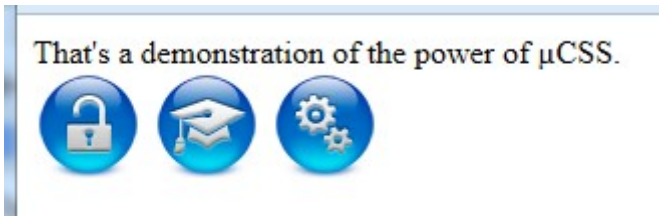
- **`buttonsAndIcons`** renders complete button and icon series including `@2x` variants from a layered PSD draft (layouts × icon glyphs) — the successor of the ButtonCreator plugin. The layer effects (drop shadow, gradients, glow, bevel, **stroke**, **satin**, etc.) are reproduced by µPS — see chapter *PSD compositor*.



Layer structure of a button PSD draft (`icons`, `layouts`, states)



The same icon glyphs, rendered with two layouts of the same draft document ("alu" and "aqua")



With `mode: "topLayerSets"` the step works in the second legacy mode (`CreateByTopLayerSets`): instead of the icon×state matrix, **each direct subgroup of the layout group becomes exactly one image**, named after the group. No `icons` group is needed here — typical for logos, animation frames (e.g. `activityindicator`) and emoticons. Optionally `setPattern` (regex string) narrows the exported groups.

```
{ buttonsAndIcons: "dev/media/final/general/activityindicator.psd", layout: "std",
```

```
  mode: "topLayerSets", outputDir: "imgs/general" }
```

- **`appIcons`** generates app icons and favicons profile-based (`web`, `ios`, `play`) from a square master — the modernized AppIconMaker.
- **`sequenceStrip`** builds a horizontal animation strip with JSON frame data from an image sequence (individual files or one image in DSD format):



Animation strip generated from 19 individual frames of a smoke animation

- **`copy`** / **`copyFolder`** copy static files (make-style: only when the source is newer).

Full reference for all image generators, DSD format, draft workflow and raster APIs: **chapter [microPS (µPS)](#microps-µps--image-generators)**.

4 microPS (µPS) — image generators

The sibling module **µPS** (`gulp-mu-ps`, display name **µPS**) handles the image-processing functions of the old Adobe workflow — **without an Adobe dependency**. µCSS™ integrates µPS via `media` steps and sprite/cursor directives; µPS can also be used **standalone** via `npm install gulp-mu-ps`.

µPS reads PSD sources via `ag-psd`; image processing uses `sharp`. Layered drafts are authored in **[Affinity](https://affinity.studio/download)** or **[Photopea](https://www.photopea.com/)** (browser, full PSD). The main layer effects are reproduced (drop shadow, gradients, glow, bevel, **stroke**, **satin**); pattern overlay and exact gradient strokes are missing — see chapter *PSD compositor*.

4.1 API overview

Export	Description
`ButtonAndIconCreator`	Button/icon series from PSD drafts (layouts × glyphs)
`AppIconMaker`	App icons/favicons profile-based (`web`, `ios`, `play`)
`SpriteAtlas`	Sprite atlas with deduplication and JSON map
`TileSheet`	Tile texture: splitting, dedupe, empty tiles
`SequenceStrip`	Horizontal animation strips from file series or DSD images
`ScanDsdImage`	DSD scanner (cells, type, frame number, anchor)
`CreateDsdFromImages`	Build DSD master sheets from frame images
`CreatePsdWithSequence`, `InsertSequenceIntoGroup`	Write frame sequences into PSD layer groups
`ApplyGamma`, `ApplyBrightnessContrast`, `ApplyHueSaturation`, `ApplyAdjustmentStack`	PS-like adjustments with optional mask
`MaskFromRects`, `VisitMaskedPixels`	Mask helpers (e.g. tile selections)
`MaskFromAlpha`, `MagicWand`, `ColorRange`, `FeatherMask`, `InvertMask`, `SelectionBounds`, `CopySelection`	Selection engine (alpha, magic wand, color range, feather, invert)
`CreateRaster`, `CloneRaster`	Empty raster / copy
`LoadRasterFromImage`, `SaveRasterAsImage`, `SaveRasterAsImageHalfSize`	Load/save images
`ScaleRaster`, `CropRaster`, `FlipRaster`, `RotateRaster`, `RotateRasterAround`, `PasteRaster`, `ResizeCanvas`	Geometry and canvas size
`MuPsDoc`	ExtendScript-like entry point (flat, phase 1)
`OpenDrafts`, `WatchDrafts`	Affinity/draft workflow (save → rebuild)
`PsDocument`, `RenderDocument`	Load and composite PSD

4.2 ButtonAndIconCreator

Expected layer structure (@2x master):

```
layouts/
  <layout>/            e.g. "std", "alu", "aqua"
    <state>/           e.g. "normal", "hover" – or "_" (see naming scheme)
```

```
      …                 background layers
      icon              placeholder carrying layer effects for icons
icons/
  <glyphName>           one pixel layer or group per glyph, e.g. "but_login_"
```

For each glyph×state combination, the **`effects`** of the `icon` placeholder are transferred onto the glyph (copy/paste layer style) — **not** the placeholder blend mode or layer opacity; the document is rendered and saved. Glyphs named `_` are skipped.

```
import { ButtonAndIconCreator } from "gulp-mu-ps";

await ButtonAndIconCreator.Create("drafts/buttons.psd", {
    layout: "aqua",
    outputDir: "imgs/aqua"
});
```

Option	Default	Effect
`layout`	— (required)	Layout group under `layouts/`; the name does **not** appear in file names — use `outputDir` to distinguish layouts.
`outputDir`	— (required)	Target directory.
`retina`	`true`	@2x master: `<name>@2x.<ext>` and `<name>.<ext>` (50 % downscale, Lanczos). With `false`, one file at document resolution.
`format`	`"png"`	`"png"` or `"webp"`.
`iconsGroupName`	`"icons"`	Glyph group name.
`layoutsGroupName`	`"layouts"`	Layouts group name.

File names: `<glyphName><stateName>[@2x].<ext>` — state appended **without separator** (`but_login_` + `normal` → `but_login_normal.png`). State `_` → no suffix (`pointer.png`). Layout names never in file names.

4.2.1 CreateByTopLayerSets — one image per group

No `icons` group needed: each direct child of the layout is a finished composition → one image named after the group (logos, animation frames, emoticons).

```
<layout>/
  <setName>/            e.g. "frame01"
    …
  _/                    skipped
```

```
await ButtonAndIconCreator.CreateByTopLayerSets("drafts/activityindicator.psd", {
    layout: "std",
    outputDir: "imgs/general",
    setPattern: /^frame/
});
```

Only the current subgroup is visible when rendering; siblings are hidden — like the legacy plugin.

4.3 AppIconMaker

Square master (PSD/PNG, ≥ 1024×1024):

```
import { AppIconMaker } from "gulp-mu-ps";
```

```
await AppIconMaker.Create("drafts/appicon.psd", {
    outputDir: "icons",
    profiles: ["web", "ios", "play"],
    layout: "aqua",
    appName: "MyApp",
    themeColor: "#0a5ae0",
    background: "#ffffff"
});
```

Profile	Produces
`web`	`favicon.ico`, PWA icons, `apple-touch-icon`, `site.webmanifest`, HTML snippet
`ios`	`appstore-icon-1024.png` (alpha removed)
`play`	Play Store icon plus adaptive foreground/background layers

Custom profiles: `{ name, icons: [{ file, size, mode }] }` with `mode`: `plain`, `flatten`, `maskable`, `background`, `ico`.

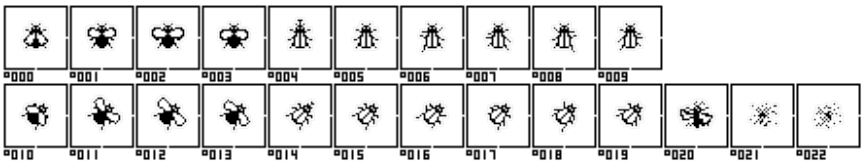
4.4 SpriteAtlas and TileSheet

`SpriteAtlas` packs many source images into an atlas (1×/@2x, optional WebP, JSON map). `TileSheet` splits sources into uniform tiles, deduplicates and writes square textures plus JSON map — foundation for Oxyd/Unity pipelines (app-specific post-processing: `docs/CONCEPT.md` D12).

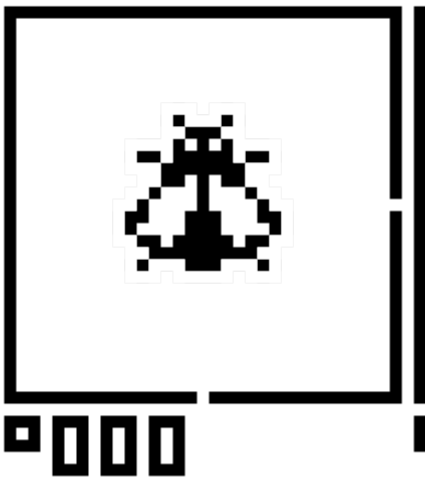
4.5 SequenceStrip and DSD format

`SequenceStrip` builds a horizontal strip plus JSON frame data from single files or a **DSD image** (legacy SpriteTools-compatible).

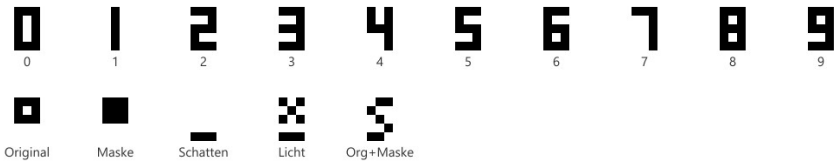
DSD ("Dongleware Sprite Definition"): all frames in **one** image, each frame in a marked cell with anchor markers and label (type glyph + three-digit frame number):



DSD source (flyex)



DSD cell magnified



DSD glyph reference

Scanner `ScanDsdImage` detects cells automatically; `SequenceStrip` orders by frame number:



Flyex strip

Trimmed content width when authoring should be **≥ 10 px** so the frame label fits inside the cell border.

4.5.1 Authoring DSD (`CreateDsdFromImages`)

Counterpart to `ScanDsdImage` — build a DSD master from frame PNGs:

```
import { CreateDsdFromImages, DSD_SIGN_ORIGINAL } from "gulp-mu-ps";

await CreateDsdFromImages(["frames/f0.png", "frames/f1.png"], {
    outputFile: "sprites/series.png",
    signType: DSD_SIGN_ORIGINAL,
    pivotPercent: { x: 0, y: 0 },   // anchor top-left of trimmed content (default)
    frameNumbers: [0, 1]
});
```

Pivot: `pivotPercent` (0…100 relative to the trimmed content rect) is the recommended form — `0/0` = top-left, `50/50` = center, `100/100` = bottom-right. Alternatively `pivot: { x, y }` in **pixel offsets** (DSD convention; negative = anchor right/below content). Each frame resolves the anchor from its own trim box.

4.5.2 PSD sequence groups

```
import { CreatePsdWithSequence, InsertSequenceIntoGroup } from "gulp-mu-ps";

await CreatePsdWithSequence(["frames/f0.png", "frames/f1.png"], {
    outputFile: "drafts/sequence.psd",
    groupName: "animation"
});
```

`PsDocument` supports `Save()` for round-trips after in-memory edits.

4.6 Raster model, I/O and canvas size

µPS works on **rasters** internally: `{ width, height, data: Float32Array }` — **RGBA 0…1**, non-premultiplied, full document area.

Export	Description
`CreateRaster(w, h)`	Empty canvas (transparent)
`CloneRaster(raster)`	Deep copy
`LoadRasterFromImage(path)`	Load PNG/WebP/JPEG/…
`SaveRasterAsImage(raster, path)`	Save (`.png` / `.webp` from extension)
`SaveRasterAsImageHalfSize(raster, path)`	@2x → 1× (50 % Lanczos)

```
import { CreateRaster, LoadRasterFromImage, SaveRasterAsImage } from "gulp-mu-ps";

const blank = CreateRaster(512, 512);
const photo = await LoadRasterFromImage("photo.jpg");
await SaveRasterAsImage(photo, "out/photo.webp");
```

Canvas size (content **not** scaled — PS *Image → Canvas Size*):

```
import { LoadRasterFromImage, ResizeCanvas, SaveRasterAsImage } from "gulp-mu-ps";

const raster = await LoadRasterFromImage("icon.png");
const padded = ResizeCanvas(raster, {
    width: 128,
    height: 128,
    anchor: "middleCenter",
    fill: [0, 0, 0, 0]
});
await SaveRasterAsImage(padded, "out/icon-padded.png");
```

Anchors: `topLeft`, `topCenter`, `topRight`, `middleLeft`, `middleCenter`, `middleRight`, `bottomLeft`, `bottomCenter`, `bottomRight`. Shrinking crops overflow.

Planned (baseline, not yet implemented): indexed colour / **CLUT** and **bit-depth reduction** (`noofbits`) for texture pipelines — see `docs/CONCEPT.md` D15.

4.7 Adjustments and masks

Headless counterpart to PS *Image → Adjustments*. All functions modify the raster **in place**; optionally limited to a **selection** (mask).

4.7.1 Masks

Form	Use
Rectangle list `[{ x, y, w, h }, …]`	Oxyd-style: many 64×64 tiles as one selection
`MaskFromRects(w, h, rects)`	0/1 weight map
`Float32Array` (length w×h)	Soft mask 0…1
`undefined` / omitted	Whole image

4.7.2 Selection engine

Low-level helpers for PS-like selections on flat RGBA rasters (RGB distance 0…1):

Export	Effect
`MaskFromAlpha(raster, { min })`	All pixels with alpha ≥ `min`
`MagicWand(raster, x, y, { tolerance })`	Connected flood-fill from seed pixel
`ColorRange(raster, { rgb, fuzziness })`	All matching pixels globally (not just connected)
`FeatherMask(mask, w, h, radius)`	Soft edges (box-blur approximation)
`InvertMask(mask)`	Invert selection
`SelectionBounds(mask, w, h)`	Bounding box or `null`
`CopySelection(raster, mask?)`	Copy selected pixels into a new raster

Via **`MuPsDoc.selection`**: `SelectAlpha`, `MagicWand`, `ColorRange`, `Feather`, `Invert`, `Bounds` — plus `Copy()`, `Paste(other, { x, y })`, `Duplicate()` on the document.

4.7.3 ApplyGamma

PS *Exposure* / Oxyd `GammCorrection(γ)`: exponent `1/γ` on RGB.

Parameter	Range	Default
`_gamma`	~0.01…9.99 (1 = no change)	—
`exposure`	PS exposure	`0`
`offset`	PS offset	`0`
`mask`	see above	whole image

4.7.4 ApplyBrightnessContrast

Parameter	Range	Default
`brightness`	−100…100	`0`
`contrast`	−100…100	`0`
`useLegacy`	linear legacy model	`false`

4.7.5 ApplyHueSaturation

Master channel only (no per-colour ranges):

Parameter	Range	Default
`hue`	−180…180 (degrees)	`0`
`saturation`	−100…100	`0`
`lightness`	−100…100	`0`

4.7.6 ApplyAdjustmentStack

Multiple steps in order, **one** mask for all steps:

```
import {
    CreateRaster,
    ApplyAdjustmentStack,
    MaskFromRects
} from "gulp-mu-ps";

const raster = CreateRaster(512, 512);
const mask = MaskFromRects(512, 512, [
    { x: 0, y: 0, w: 64, h: 64 },
    { x: 64, y: 0, w: 64, h: 64 }
]);
ApplyAdjustmentStack(raster, [
    { type: "gamma", gamma: 1.2 },
    { type: "brightnessContrast", brightness: 10, contrast: 5 },
    { type: "hueSaturation", saturation: -15 }
], { mask });
```

Visual closeness to PS, not bit-exact — tune against reference PNGs where needed.

4.8 Raster transforms

Geometry on the raster — scaling uses `sharp` kernels.

Export	Effect
`ScaleRaster`	Scale: `scale`, or `width`/`height`, `kernel`
`CropRaster`	Rectangle `{ x, y, w, h }` (clamped to bounds)
`FlipRaster`	`"horizontal"` / `"vertical"`
`RotateRaster`	90°, 180°, 270°
`RotateRasterAround`	Free angle, pivot `{ x, y }` (bilinear)
`PasteRaster(cw, ch, raster, { x, y })`	Place raster on larger blank canvas
`ResizeCanvas`	Canvas size (see above)

```
import {
    LoadRasterFromImage,
    ScaleRaster,
    CropRaster,
    RotateRasterAround,
    SaveRasterAsImage
} from "gulp-mu-ps";

const raster = await LoadRasterFromImage("photo.jpg");
const half = await ScaleRaster(raster, { scale: 0.5, kernel: "lanczos3" });
const patch = CropRaster(half, { x: 100, y: 50, w: 640, h: 320 });
const tilted = RotateRasterAround(patch, 15, { x: 0, y: 0 });
await SaveRasterAsImage(tilted, "out/patch.png");
```

ScaleRaster kernels: `nearest`, `linear`, `cubic`, `lanczos2`, `lanczos3` (default). PS `ResampleMethod` approximations: `BICUBICSHARPER` → `lanczos3`, `BILINEAR` → `linear`, `NEARESTNEIGHBOR` → `nearest`.

Visual compare: `node gulp-mu-ps/tools/verify-image-ops.mjs <image> <outdir>`.

4.9 ExtendScript entry point (`MuPsDoc`)

Optional **compatibility wrapper** for JSX/Oxyd habits — **not** a PS DOM, no `executeAction` emulation. Maps familiar names onto the raster engine (phase 1: flat images).

ExtendScript / Oxyd	`MuPsDoc` / µPS
`app.activeDocument` (flat)	`await MuPsDoc.Open(path)`
`doc.width` / `doc.height`	`.width` / `.height`
`selection.select(rects)`	`doc.selection.Select([…])`
Alpha / magic wand / color range	`SelectAlpha`, `MagicWand(x,y)`, `ColorRange({ rgb, fuzziness })`, `Feather`, `Invert`, `Bounds`
`GammCorrection(γ)`	`doc.GammaCorrection(γ)`
Brightness/contrast	`doc.BrightnessContrast(b, c)`
Hue/saturation	`doc.HueSaturation({ … })`
`doc.resizeImage(…)`	`await doc.ResizeImage({ scalePercent, method: "BICUBICSHARPER" })`
Canvas size	`doc.CanvasSize({ width, height, anchor })`
`doc.saveAs(…)`	`await doc.SaveAs(path)`
@2x + 50 % downscale	`await doc.SaveAsRetinaPair(dir, name)`
Copy / paste / duplicate	`doc.Copy()`, `doc.Paste(other, { x, y })`, `doc.Duplicate()`

```
import { MuPsDoc } from "gulp-mu-ps";

const doc = await MuPsDoc.Open("atlas.png");
doc.selection.MagicWand(12, 8, { tolerance: 0.08 });
doc.GammaCorrection(1.2);
const patch = doc.Copy();
await doc.SaveAs("out/atlas-gamma.png");
```

Phase 2 (planned): PSD (`OpenPsd`), `findLayer`, `copyLayerStyle`, `flatten` — see `docs/CONCEPT.md` D14.

The **low-level API** (`ApplyGamma`, `ScaleRaster`, …) remains the engine; `MuPsDoc` is a migration aid only.

4.10 PSD compositor and layer compositing

Under the API surface, **`PsDocument`** / **`RenderDocument`** and **`Compositor.mjs`** composite loaded PSDs: visible layers, groups, text, layer effects, blend modes. This powers **ButtonAndIconCreator** (style transfer) and direct rendering.

Visually close to Photoshop/Affinity, not bit-exact — drift is monitored via **MAE** regression against Adobe reference PNGs (`test/ReferenceRender.test.mjs`).

4.10.1 Reproduced layer effects (layer styles)

µPS reads layer effects from the PSD (`ag-psd`, field `effects`) and renders them in **`Effects.mjs`**. Visually close to Photoshop/Affinity, **not bit-exact** — drift is monitored via MAE regression against Adobe reference PNGs.

Supported

Photoshop / Affinity	Key (`ag-psd`)	Notes
Color overlay	`solidFill`	incl. opacity and **per-effect blend mode**
Gradient overlay	`gradientOverlay`	**linear only** (angle from PSD); radial/angle/reflected not

Photoshop / Affinity	Key (`ag-psd`)	Notes
		supported
Inner glow	`innerGlow`	size, spread (choke), **range**, **contour curve**
Outer glow	`outerGlow`	as inner; visible outside the shape only
Inner/drop shadow	`innerShadow`, `dropShadow`	distance, angle, size, spread, color, blend mode
Bevel and emboss	`bevel`	styles: inner bevel, outer bevel, pillow emboss, emboss; smooth vs. chisel hard; **contour curve** on bevel; separate highlight/shadow colors and blend modes
Stroke	`stroke`	position inside / center / outside; solid color (simple gradient via first stop); **no** pattern stroke
Satin	`satin`	distance, size, angle, invert, contour curve, blend mode

Contour curve vs. stroke effect: In effect dialogs, the *Contour* tab is the **falloff curve** for glow, bevel, and satin — µPS reads and applies it. The **Stroke** layer style is listed above as `stroke`.

Not implemented (ignored)

Photoshop / Affinity	Key	Workaround
Pattern overlay	`patternOverlay`	pattern as a pixel layer (`ag-psd`: Patt section limited)
Pattern stroke	`stroke` (`fillType: pattern`)	solid stroke or pixel layer
Other gradient types	—	linear only in `gradientOverlay`; stroke gradient approximated (first color stop)

Blend modes (layers and effects)

Each **layer** and each **individual effect** can carry its own blend mode (the *Mode* control in the effect dialog). µPS uses the same table in **`BlendModes.mjs`** — for layer compositing and for mixing effect colors (e.g. inner shadow with *Color Burn*).

English (`ag-psd`)	German (Photoshop)	µPS
normal	Normal	✓
multiply	Multiplizieren	✓
screen	Abwedeln	✓
overlay	Überlagern	✓
darken	Abdunkeln	✓
lighten	Aufhellen	✓

English (`ag-psd`)	German (Photoshop)	µPS
color burn	Farbig nachbelichten	✓
color dodge	Farbig abwedeln	✓
linear burn	Linear abwedeln (often labelled "Negativ multiplizieren")	✓
linear dodge	Linear abwedeln (Add)	✓
hard light	Hartes Licht	✓
soft light	Weiches Licht	✓
difference	Differenz	✓
exclusion	Ausschluss	✓
pass through	Pass through (groups)	✓

Not supported (fallback: normal): e.g. darker/lighter color, vivid/linear/pin light, hard mix, hue, saturation, color, luminosity, divide, subtract.

Reference matrix in the PSD: `layouts/blend/` → PNGs under `examples/reference/out/blend/` (normal, multiply, screen, overlay, darken, lighten, color burn).

4.10.2 Style transfer (ButtonAndIconCreator) vs. layer links

Two different concepts — often confused:

Mechanism	Where	What happens
Style copy (CpFX/PaFX)	`ButtonAndIconCreator`	The full `effects` **block** of the `icon` placeholder is applied to the glyph (legacy Photoshop: copy/paste layer style).
Layer link (PS *Link Layers*)	Authoring in PS/Affinity	**Move together** in the editor only — no separate render mode in µPS.

What style transfer does not copy (same as the legacy plugin): the placeholder's **blend mode**, **layer opacity**, and **position** — the glyph keeps its own values. Per-effect blend modes inside `effects` (e.g. inner shadow → *Color Burn*) **are** transferred.

The reference PSD includes `layouts/links/` with linked layers (`icon_master`, `follower`) for flat compositing regression — not a separate "link mode" feature.

4.10.3 Opacity model (three independent controls)

Control	PSD source	Effect in µPS
Pixel alpha	Per-pixel alpha in `imageData`	Shape mask for effects + compositing alpha
Fill opacity	`fillOpacity` (iOpa), 0…1	Scales **fill alpha only**; layer styles stay full strength
Layer opacity	`opacity`, 0…1	Scales the **entire** layer (fill + styles) at composite time

Rainbow discs (not flat fills) in `compositing/` expose channel/blend bugs that solid fills hide. Synthetic tests: `test/Compositing.test.mjs`.

4.10.4 Reference PSD and Adobe ground truth

Development and tuning use the comprehensive reference under `gulp-mu-ps/examples/reference/`:

Path	Role
`examples/drafts/mups-reference.psd`	Reference PSD (compositor, effects, blend, text, both ButtonAndIconCreator modes)
`examples/reference/out/`	Adobe-export PNGs (ground truth)
`tools/photoshop/build-mups-reference.jsx`	Build PSD
`tools/photoshop/export-mups-reference.jsx`	Export PNGs (re-runs skip existing files)
`tools/build-reference-overview.mjs`	Contact sheet `out/overview/mups-reference-sheet.png`

Isolated effect layouts (`layouts/fx/`): one preset per layer-style variant — e.g. `solidFill`, `dropShadow` (3 variants), `innerShadow` (4), `innerGlow`/`outerGlow` (3 each), `gradientOverlay` (3), **`strokeOutside`**, **`strokeInside`**, **`strokeCenter`**, **`strokeMultiply`**, **`satin`**, **`satin_warm`**, **`satin_invert`**, `bevelInner` (5 bevel variants). Each layout yields Adobe PNGs under `out/fx/` (style transfer onto `glyph_disc`) and `out/flat/fx/` (direct compositing without CpFX/PaFX).

Tuning hints: Combined stacks (`stacks/stack_aquaIcon`) for production-like checks; isolated pairs for parameter sensitivity — e.g. `dropShadow` vs. `dropShadow_soft`, `bevelInner` vs. `bevelInner_chisel`, **`strokeOutside` vs. `strokeMultiply`**, **`satin` vs. `satin_invert`**. After changes to `Effects.mjs` or JSX presets: rebuild PSD → export → `npx gulp reference:render` → `npm test` (µPS).

Developer workflow: `buttons.psd` → JSX build → export → render µPS with `retina: false` → `compare-images.mjs` or `ReferenceRender` tests. Details: `gulp-mu-ps/examples/reference/README.md`.

Not in this PSD: SpriteAtlas, TileSheet, SequenceStrip/DSD, AppIconMaker — separate legacy references under `oldsrcs/…/examples/` (PNG/sequence pipelines).

4.10.5 Tools (µPS development)

- `node tools/inspect-psd.mjs <file.psd>` — layer tree and effects
- `node tools/compare-images.mjs <a> <b>` — PNG compare (MAE)
- `node tools/open-drafts.mjs` / `watch-drafts.mjs` — draft workflow from the shell

4.11 Draft workflow (authoring + watch)

Authoring in **Affinity** or **Photopea** — neither replaces µPS rendering. The automation path:

1. Edit and save PSD (desktop or browser).
2. `WatchDrafts` or `gulp.watch` detects the change (debounce ~1.5 s).
3. µPS renders headless; µCSS™ build runs when needed.

```
import { WatchDrafts, OpenDrafts } from "gulp-mu-ps";

WatchDrafts(["dev/media/final/panelbuttons.psd"], async () => {
    await BuildSkin("skins/src/std.µcss.mjs");
}, { debounceMs: 1500 });

OpenDrafts("dev/media/final/panelbuttons.psd", { app: "affinity" });
```

Environment: `MU_AFFINITY_EXE`, `MU_DRAFT_APP`. Details: `docs/CONCEPT.md` (D11).

4.11.1 Intermediate steps raw → final (`outputBase`)

By default all steps write into the skin output directory. With `outputBase: "project"` a step writes relative to the project root instead — this lets you express the generation of intermediate results (e.g. sequence strips from `dev/media/raw/...` into `dev/media/final/...`) directly in the manifest. The steps run in manifest order, and a following `copy`/`copyFolder` step then takes the result into the skin like any other final asset:

```
media: [
    // 1. generate the strip from the raw frames into the final tree ...
    { sequenceStrip: "dev/media/raw/flyex/frames", outputFile:
"dev/media/final/general/gui/flyex/flyex.png", outputBase: "project" },
    // 2. ... and copy it from there into the skin as usual.
    { copyFolder: "dev/media/final/general/gui/flyex", to: "imgs/general/gui/flyex",
filter: "\\.(png|json)$" }
]
```

The incremental build applies to these steps too: generation only happens when the result is missing or the configuration or sources (mtime/size) have changed. Alternatively, the raw → final step can of course still run as its own Gulp task before the µCSS™ build — `outputBase: "project"` is the way to go when everything should live in a single manifest file.

5 microAU (µAU) — sound atlas

The sibling module **µAU** (`gulp-mu-au`) builds a **sound atlas** from many short audio files — the audio counterpart of sprite atlases: one combined WAV/MP3 blob plus a JSON timing map for the browser runtime (e.g. µLib `Sounds`).

Scope: build layer only. Referencing sounds *from CSS* (µCSS™ sound directive) and browser runtime/speech follow later.

Engine: `gulp-mu-sound-atlas` (decode → resample → optional MP3). µAU is the µPS-style wrapper with async `Create` and incremental cache.

5.1 API overview

Export	Description
`SoundAtlasMaker`	Collect sources → build atlas + JSON (cached)
`ListAudioFiles`	Recursively list `*.wav`/`*.mp3`
`CONVERT`	Sample rate, sample size, channel, stereo constants

5.2 SoundAtlasMaker

```
import { SoundAtlasMaker } from "gulp-mu-au";

const result = await SoundAtlasMaker.Create({
    src: "dev/media/final/sounds",
    dataFile: "skins/std/snds/app.sounds.wav",
    jsonFile: "skins/std/snds/app.sounds.json"
});
// result => { skipped, sounds, dataFile, jsonFile }
```

Option	Default	Effect
`dataFile`	— (required)	Audio blob (`.wav` or `.mp3`)
`jsonFile`	— (required)	JSON timing map
`sounds` / `src`	—	Explicit list or recursive source directory; at least one required
`format`	from extension	`"wav"` or `"mp3"`
`mp3KBitRate`	`128`	MP3 bit rate
`sampleRate`, `sampleSize`, `channels`, `stereo`	`CONVERT.*_HIGHEST`	Target format; `*_HIGHEST`/`*_LOWEST` derive from inputs
`cacheFile`	`<dataFile>.cache.json`	Cache marker, or `false`
`force`	`false`	Force rebuild

5.2.1 Loop points in file names

`engine.ls.100.le.400.wav` → sound name `engine`, loop from sample 100 to 400. The base name without the `.ls.…le.…` suffix is the map key.

5.2.2 JSON format

```
{ "sounds": { "click": [startSec, durationSec, loopStartSec, loopEndSec], … } }
```

`-1` for `loopStart`/`loopEnd` = no loop. Consumed directly by µLib `Sounds`.

5.2.3 MP3 inputs

For **MP3 source files**, the engine may need a `patch-package` patch in the consuming project (see `gulp-mu-sound-atlas` README). WAV inputs and MP3 *output* are unaffected.

5.3 µCSS™ integration (manifest `sounds`)

µCSS™ invokes µAU via an optional **`sounds`** block in the skin manifest (parallel to `media`):

```
export default DefineSkin({
    sounds: {
        src: "dev/media/final/sounds",
        dataFile: "snds/app.sounds.wav",
        jsonFile: "snds/app.sounds.json"
    },
    files: [ … ]
});
```

Paths in `dataFile`/`jsonFile` are relative to the skin output directory; `src` relative to the project root. The build report includes `report.sounds[]` with `skipped` and the sound list. Incremental cache like µPS media steps.

Alternatively, use `copyFolder` for pre-built atlases.

5.3.1 Planned: CSS sounds, auto-wiring & handler overrides

The **build layer** (atlas + JSON timing map including loop points per sound) is implemented. **Loop yes/no** lives in `sounds[name]` — not in bindings. Bindings only define **when** to play/stop (`mode`: `oneshot` vs. `sustain`).

Pipeline (fixed): Build delivers **data** (`sounds` + `bindings[]` in JSON) — from CSS and optionally `soundTriggers.mjs` (runs **only** in Node). Runtime delivers **behavior**: first `Sounds.installBindings` (fully automatic), then optionally `soundHandlers.mjs` (patches default handlers in the browser — **without** changing JSON, **without** running at build time). `handlers` does not replace `triggers`: new events → CSS/`triggers`; different reaction → `handlers` or app JS.

Next stages:

1. **Declarative CSS** — aural properties (`cue-before`, `cue-after`, `play-during`) and directive `-µ: Sound(...)` → entries in `bindings[]` (JSON sidecar).
2. **Automatic (µLib)** — `Sounds.installBindings(json)` installs all DOM/animation listeners; no manual event JS when CSS + `triggers` suffice.
3. **Manifest `soundTriggers.mjs`** (`sounds.triggers`) — build time: extend `bindings[]` (helper selectors, FlyEx animations). **Not** `helpers.mjs`.
4. **Manifest `soundHandlers.mjs`** (`sounds.handlers`, optional) — runtime: **wrap/replace** default handlers (super call to the default method), e.g. FlyEx click logic alongside auto-wiring.
5. **App JS** — still `Sounds.play`/`Sounds.stop` for free logic without bindings.

Details: `gulp-mu-au/docs/CONCEPT.md` (§5–§8, section “Pipeline”). FlyEx demo today: everything in `demo.js`; target: loops via `triggers` + auto-wiring, edge cases via `handlers`.

6 microFT (µFT) — icon font

µFT (`gulp-mu-ft`) builds an **icon font** from SVG glyphs — automated like IcoMoon, without Adobe/IcoMoon — and packages **existing webfonts**, including modern OpenType color fonts and emoji fonts. Output: font files (SVG/TTF/EOT/WOFF/WOFF2), CSS (`@font-face` and icon classes), IcoMoon-compatible JSON (icons), HTML overview.

Requires **Node ≥ 20.11**. Built on `svgicons2svgfont`, `svg2ttf`, `ttf2woff(2)`, `ttf2eot`. The SVG icon path stays **monochrome**. Color information is preserved when `src` is an existing TTF/OTF/WOFF/WOFF2 file.

6.1 Glyph naming

```
<name>-U0x<HEX>.svg
e.g. general-control-edit-U0xE900.svg  →  name "general-control-edit", code U+E900
```

- **Directory** = group id (relative under `src`, e.g. `general` or `medical/diagnosis`)
- Without valid `-U0x<HEX>` → skipped (drafts)
- Duplicate codepoints → warning, conflicting files named

6.2 FontGenerator

```
import { FontGenerator } from "gulp-mu-ft";

const result = await FontGenerator.Create({
    fontName: "AppSymbol",
    src: "svg",
    outputDir: "fonts",
    groups: {
        general: { label: "General", description: "General UI controls.", order: 1 }
    },
    glyphs: {
        "general-control-edit": { description: "Button/icon to start editing." }
    }
});
// result => { skipped, glyphs, warnings, files }
```

Produces e.g. `AppSymbol.css` (`.icon-<name>`), `AppSymbol.json`, `AppSymbol.html`.

Option	Default	Effect
`fontName`, `src`, `outputDir`	— (required)	Font name, SVG tree **or** a single TTF/OTF/WOFF/WOFF2 file, target
`formats`	all common	Font extensions to emit
`fontHeight`	`1024`	Em height
`normalize`, `centerHorizontally`	`true`	passed to `svgicons2svgfont`
`classPrefix`	`"icon"`	CSS class prefix
`fontUrlBase`	`""`	URL prefix in CSS/HTML
`css`, `json`, `html`	enabled	control output or `false`
`groups`, `glyphs`	`{}`	HTML overview metadata (**English only**)
`timestamp`	fixed	reproducible font timestamps
`cacheFile`	`.build-cache.json`	content signature (SHA-1),

Option	Default	Effect
		timestamp-independent
`force`	`false`	force rebuild

6.3 API building blocks

Export	Description
`FontGenerator`	scan → build font → write CSS/JSON/HTML
`ScanGlyphs`	scan directory, derive codepoints/groups
`BuildFontFormats`, `ConvertFontFormats`, `BuildFontCss`, `BuildWebfontCss`	individual build steps
`InspectFont`, `DetectColorFont`	color-table detection
`ListSvgFiles`, `ReadGlyphSvg`	low-level helpers

6.4 Color fonts and emoji fonts

Point `src` at a color font file instead of an SVG directory — same `FontGenerator.Create` call. µFT detects COLR/CPAL (v0 and v1), SVG-in-OpenType, CBDT/CBLC and sbix, wraps TTF/OTF as WOFF/WOFF2 **without dropping those tables**, and emits a standard `@font-face` (no icon classes).

```
await FontGenerator.Create({
    fontName: "ChatEmoji",
    src: "vendor/NotoColorEmoji-Regular.ttf",
    outputDir: "dev/media/final/fonts"
});

body { font-family: ChatEmoji, system-ui, sans-serif; }
/* Optional palette: font-palette / @font-palette-values — µFT does not rewrite CPAL. */

<p>Hello 😀 🎉</p>
```

Color rendering depends on the font, the browser and the OS. µFT does **not** generate emoji animation. EOT and legacy SVG fonts are skipped for color sources (they cannot carry color tables). Subsetting is not implemented; the full font is packaged so COLR layer glyphs stay intact. Details and the detected/preserved/converted matrix: `gulp-mu-ft` README, section *Color Fonts*. Example: `gulp-mu-ft/examples/color-font/`.

6.5 µCSS™ integration

Manifest (planned): optional `font` block in chapter *Manifest reference* — `src` (SVG directory or color/emoji font file), `include` (single SVGs), `outputDir`, metadata; builds via µFT before CSS compilation.

Today: typically a **separate Gulp step** before the skin build, then `copyFolder` in the manifest:

```
media: [
    { copyFolder: "dev/media/final/fonts", to: "fonts", filter: "\\.(woff2?|ttf|css)$" }
]
```

Also planned: CSS directive `-µ: Glyph("icons/edit.svg")` for single SVGs in rules (symmetric to `Sprite()`/`Sound()`). Reference icon fonts in `.µ.css` via `@font-face` and classes (codepoint variables in `vars`, e.g. `icon_pencil: "\\e91c"`).

7 Vue and component co-location

Component frameworks like Vue usually keep styles in `<style scoped>` inside the `.vue` file. That works, but every component gets its own `data-v-…` attribute, the compiled CSS grows quickly and live editing in the browser DevTools becomes awkward across dozens of scoped blocks.

µCSS™ uses a different approach: **co-location without Vue processing the styles**. Each component keeps a style file **next to** its `.vue` file, but with a private extension such as `*.π.css` (Greek pi — Vite and Vue ignore it). One main stylesheet pulls them all in at build time; µCSS™ bundles everything into a single, static CSS file.

7.1 Workflow

1. Place component styles in `MyButton.π.css` beside `MyButton.vue`.
2. In the main entry (e.g. `main.µ.css`) collect them with a build-time import:

```
@import "components/**/*.π.css";
body { margin: 0; }
```

3. Enable rule merge in the manifest when many components share selectors (e.g. `.btn`):

```
export default DefineSkin({
    merge: { onConflict: "error" },
    files: [{ source: "main.µ.css", target: "app.css" }]
});
```

4. In the Vue template use **plain class names** — no `<style scoped>`, no `data-v-…`. The browser loads one real CSS file; DevTools stay fully usable.

7.2 Namespaces and global state classes

When two components both define `.card`, enable merge (above) or avoid collisions with `@µ-namespace` per component file:

```
@µ-namespace MyButton;
.btn { padding: 10px; }
.btn:global(.is-active) { box-shadow: 0 0 0 2px #99bbff; }
```

Only classes in that file are prefixed (`.MyButton-btn`). State classes toggled from JavaScript (`.is-active`, shared utilities) stay global via `:global(...)`.

7.3 Migrating existing Vue SFCs

The repository includes a migration aid analogous to the LESS converter. It extracts `<style>` blocks from `.vue` files, writes the sidecar `ComponentName.π.css`, injects `@µ-namespace`, strips scoped `[data-v-…]` selectors and removes the `<style>` block from the SFC (an HTML comment points to the sidecar):

```
npx gulp convert:vue
# or: VUE_IN=gulp-mu-css/examples/vue VUE_OUT=out npx gulp convert:vue
```

Example sources live under `gulp-mu-css/examples/vue/`. The tool also writes `main.µ.css` with `@import "**/*.π.css";` and a manifest skeleton including `merge.onConflict: "error"`. Review warnings before use — `lang="scss"`, CSS modules and `v-bind()` in CSS require manual follow-up; `lang="less"` is piped through the LESS converter first.

8 Core ideas of µCSS™

8.1 The .µ.css format

Source stylesheets are named `*.µ.css`. The double suffix makes editors recognize the files as regular CSS, so syntax highlighting works without further configuration. The extension is irrelevant to the compiler — the manifest references the sources explicitly; anyone who wants to avoid the µ character in file names can use `*.mu.css` instead. Content-wise, a `.µ.css` file is standard CSS with exactly two extension points — both are syntactically valid CSS, so editors and linters work normally:

1. **Value interpolation** `µ(expression)` — wherever a CSS value can appear.
2. **Directives** `-µ: expression;` — as a declaration inside a rule.

Anyone without the µ character on their keyboard uses the ASCII aliases `mu(...)` and `-mu:` — both forms are equivalent.

Unlike the old µCSS™, there are **no control rules** (`::-µcss-init` etc.) in the stylesheet any more: everything that controls compilation (variables, cursor definitions, media generation, file mapping) lives in the skin manifest — a regular JavaScript file (see below).

8.2 Value interpolation µ(expression)

From a CSS point of view, `µ(...)` is an unknown but valid function. During compilation its content is evaluated as a JavaScript expression and the `µ(...)` occurrence is replaced by the result:

```
::selection {
    background-color: µ($.selectBaseBrgdOnDarkColor);
    color: µ($.selectBaseTextColor);
}
```

Multiple interpolations per value are allowed (`padding: µ($.padY)px µ($.padX)px;`), as are interpolations in at-rule parameters (e.g. `@media (max-width: µ($.breakpoint)px)`). If an expression returns `null` or `undefined`, compilation aborts with an error message — so typos in variable names show up immediately.

This single extension replaces the entire `Set*` family of the old µCSS™ (`SetColor`, `SetBackgroundColor`, `SetZIndex`, `SetWidth`, `AddProperty` for simple values, etc.): the property is back in its place, and a placeholder value is no longer needed.

8.3 Directives -µ: expression

Directives are declarations with the property name `-µ` (or `-mu`). Their value is a single JavaScript expression executed with access to the surrounding rule and the document. The directive itself is removed from the output:

```
div.panel.modal div.companylogo {
    -µ: Sprite("imgs/logos/company_logo.png");
    margin-left: auto;
}
*.cursor_zoom {
    -µ: Cursor("zoom");
}
div.content.glittery {
    -µ: Sprite("imgs/general/gui/glittery/glittery.png", { afterWork: GlitterySprite });
}
div.carbon {
    -µ: DataUri("imgs/general/gui/carbon.png");
}
```

The `µ.` prefix of the old µCSS™ is gone — the context is implicit. Directives are evaluated in document order.

8.4 The skin manifest

Per skin (CSS theme) there is a manifest file `<skinname>.µcss.mjs` in the source directory — regular JavaScript (ES6+), importable and testable. The double suffix `.µcss.mjs` (ASCII alternative `.mucss.mjs`) clearly marks the file as a µCSS™ skin manifest so it is not confused with an ordinary module or gulpfile; the skin name is the part before it (`std.µcss.mjs` → skin `std`). It fully replaces the old control file `µ.std.css`:

```
// skins/src/std.µcss.mjs – skin "std", target: skins/std/
import { DefineSkin } from "gulp-mu-css";
import { GlitterySprite, FlyEx, Borders, TableBackgrounds } from "./helpers.mjs";

export default DefineSkin({
    vars: {
        baseBrgdColor: "#202020",
        selectBaseBrgdColor: "#007570",
        PanelZIndex: 9000
    },
    helpers: { GlitterySprite, FlyEx, Borders, TableBackgrounds },
    cursors: [
        { name: "zoom", fallback: "zoom-in", image: "imgs/general/gui/cursors/zoom.png",
hotspot: [10, 8] }
    ],
    media: [
        { buttonsAndIcons: "dev/media/final/general/gui/panelbuttons.psd", layout: "std",
outputDir: "imgs/general/gui/panelbuttons" },
        { copyFolder: "dev/media/final/fonts", to: "fonts" }
    ],
    imageFormat: "png",
    sprites: { file: "imgs/sprites.png", retina: true, preloadRule: true },
    files: [
        { source: "src.µ.css", target: "std.css" },
        { source: "src_tinymce.µ.css", target: "std_tinymce.css" }
    ]
});
```

The skin name is derived from the manifest file name, the output directory from the skin name. The complete field reference is in the chapter "Manifest reference".

8.5 JavaScript without substitute characters

In µCSS™ 1, the characters `{`, `}` and `;` had to be replaced by `«`, `»` and `¡` in JavaScript statements to avoid CSS syntax conflicts. This pattern is gone in version 2:

- `µ(...)` and `-µ: ...` contain **single expressions** — the parser counts parentheses and accounts for strings, so object literals (`{ afterWork: ... }`) and nested calls work without trouble.
- **Multi-line logic** (loops, function definitions) does not belong in the stylesheet but in the manifest's helper modules — real `.mjs` files with syntax highlighting, linting and a debugger.

9 Build process and Gulp integration

The old µCSS™ was operated through a modal dialog window in Photoshop. `BuildSkin` and Gulp take its place: the build is a regular, scriptable Node process — suitable for watch mode, CI and automation.

9.1 Directory conventions

For a manifest `skins/src/std.µcss.mjs` the following applies (all paths overridable):

Path	Meaning
`skins/src/`	Source directory: manifests, `.µ.css` files, helper modules
`skins/std/`	Output directory of the "std" skin: CSS, images, fonts, sounds
`skins/std/.cache/build.json`	Build cache of the skin (fingerprints, atlas positions)
Project root	Two levels above the manifest; base for `media` source paths like `dev/media/...`

`BuildSkin(manifestPath, options)` accepts `{ outputDir, rootDir, force }` for overriding.

9.2 Compilation flow

A `BuildSkin` run performs five steps:

1. media steps: all images are generated or copied (microPS) — they must exist before the atlas and cursor checks run.

2. Compilation: each `files` source is parsed (PostCSS); directives and interpolations are evaluated in document order. `Sprite()`/`Cursor()` calls initially only register their image references; **`DataUri()` inlines the file immediately** during this step (requires `baseDir` = skin output directory).

3. Sprite atlas: all registered images are packed (incl. `@2x`); afterwards the affected rules are rewritten.

4. Hooks: `afterWork` callbacks run with the atlas result and AST access.

5. Output: the compiled CSS files are written into the skin directory (optionally minified via `minify`), the cache is saved.

9.3 Incremental build and cache

Each run only regenerates what actually changed. The primary mechanism is file modification timestamps (`mtime` plus file size) — orders of magnitude faster than content hashes for large PSD sources:

- **Generator steps** (`buttonsAndIcons`, `appIcons`, `sequenceStrip`) are skipped when their configuration is unchanged, all source files have unchanged fingerprints and all output files still exist. PSD rendering is the most expensive part of the build — this is where the cache pays off the most.
- **`copy`/`copyFolder`** work make-style: copying happens only when the source is newer than the target.

- **The sprite atlas** is only repacked when the set of images or a source image (incl. `@2x`) has changed — otherwise the stored positions are reused and only the CSS rules are rewritten with them.
- **CSS compilation** itself is cheap and always runs when in doubt.

The cache lives per skin in `<outputDir>/.cache/build.json` and carries the version numbers of µCSS™ and the cache schema; on a mismatch it is discarded (full build). `BuildSkin(manifest, { force: true })` or deleting `.cache/` forces a full build explicitly.

9.4 Gulp tasks and watch

One task per skin is enough; `BuildSkin` is re-entrant:

```
import gulp from "gulp";
import { BuildSkin } from "gulp-mu-css";

export async function SkinStd() {
    const report = await BuildSkin("skins/src/std.µcss.mjs");
    console.log(`${report.skin}: ${report.files.length} files, atlas $
{report.atlasSkipped ? "from cache" : "repacked"}, ${report.duration} ms`);
}
export function SkinWatch() {
    gulp.watch(["skins/src/**/*.µ.css", "skins/src/*.mjs", "dev/media/final/**"], SkinStd);
}
```

The return value of `BuildSkin` is a report with `skin`, `outputDir`, `media` (step results with a `skipped` flag), `files`, `atlas`, `atlasSkipped`, `prunedSources` (deleted sprite source URLs when `sprites.pruneSources` is set), `keptSources` (protected sources when `sprites.pruneKeep` is set), `minified` (`true` when `minify` was active) and `duration`.

10 Manifest reference (DefineSkin)

`DefineSkin(configuration)` declares the skin configuration (the manifest's default export) and validates the basic structure. All fields are optional unless stated otherwise.

10.1 vars

Object with the skin variables — the replacement for `µ.$.*`. Accessible in `.µ.css` expressions as `$.name`, in helper functions as `this.$.name`. Values are arbitrary JavaScript values (strings, numbers, objects, even computed ones).

```
vars: { baseBrgdColor: "#202020", PanelZIndex: 9000, icon_pencil: "\\e91c" }
```

10.2 helpers

Object with user-defined functions (macros and hooks). They are available in `.µ.css` expressions under their name. Functions declared with `function` (not as an arrow function) receive `this` = the evaluation scope when called (see the chapter "µ evaluation context").

```
import { Borders, GlitterySprite } from "./helpers.mjs";
// ...
helpers: { Borders, GlitterySprite }
```

Note: Helpers = **CSS at compile time**. Planned: `sounds.triggers` (build → JSON data) and `sounds.handlers` (runtime → patch behavior, optional) — strictly separate; see chapter *microAU*, section “Pipeline”.

10.3 cursors

List of cursor definitions — the replacement for `µ.DefCursor`:

Field	Default	Meaning
`name`	— (required)	Name under which the cursor is used via `Cursor("name")`.
`fallback`	= `name`	Standard cursor name (W3C) as a fallback.
`image`	`""`	Image URL relative to the skin directory; empty = definition only as a fallback mapping.
`hotspot`	`[0, 0]`	Click point `[x, y]` in the image; `0,0` is omitted in the output.
`forceFallback`	`false`	Appends the fallback additionally as the last `cursor` declaration.

Cursor images are added to the preload list automatically.

10.4 media

List of media steps; they run in the given order before compilation. The step type is determined by the key field:

Step	Required fields	Further fields	Effect
`buttonsAndIcons`	source PSD, `layout`,	`retina` (default `true`),	Render button/icon

Step	Required fields	Further fields	Effect
	`outputDir`	`format`, `mode`, `setPattern`	series from a draft document (microPS ButtonAndIconCreator). `mode: "topLayerSets"` switches to "one image per top subgroup" (legacy `CreateByTopLayerSets`, no `icons` group — for logos, animation frames, emoticons); `setPattern` (regex string) filters the exported groups.
`appIcons`	source PSD/PNG	`outputDir`, `profiles`, `layout`, `background`, `appName`, `shortName`, `themeColor`	Generate app icons/favicons profile-based (microPS AppIconMaker).
`sequenceStrip`	source (folder or DSD image), `outputFile`	`retina` (default `true`), `writeMapFile` (default `true`), `format`	Generate a horizontal animation strip plus JSON frame data (microPS SequenceStrip).
`copy`	source file	`to` (target folder, default skin root)	Copy a single file when the source is newer.
`copyFolder`	source folder	`to` (default = folder name), `filter` (regex string, e.g. `"\\.(woff2?|ttf)$"`)	Copy a folder recursively (make-style), optionally filtered.

Source paths are relative to the project root, target paths relative to the skin directory. Each step additionally accepts `outputBase: "skin"` (default) or `"project"`: with `"project"` the target path is relative to the project root — for intermediate steps like raw → final (see the chapter "Automatic image generation").

10.5 imageFormat

Global image format switch: `"png"` (default) or `"webp"`. **Scope:**

- **Image-generating media steps** (`buttonsAndIcons`, `appIcons`*, `sequenceStrip`) and the **sprite atlas** (unless `sprites.format` is set, see below).
- Individual steps can override the format via their own `format` field.
- Directly referenced existing images (`url(...)` in the sources, `copy`/`copyFolder` steps) are left untouched — a `copyFolder` `filter` only copies what it lets through. If the `filter` excludes the active `imageFormat` extension (e.g. `"\\.(png|json)$"` with `imageFormat: "webp"`), the build emits a **warning** instead of silently discarding the freshly generated files.

`appIcons` produces platform-specific icon sets (PNG, partly ICO) and **deliberately ignores `imageFormat` — app/favicon formats are fixed.*

10.6 sprites

Options of the sprite atlas — the replacement for `µ.options.sprites.*`:

Field	Default	Meaning
`file`	`"imgs/sprites.png"`	Atlas file (URL as it appears in the CSS).
`format`	(none)	Atlas output format (`"png"`/`"webp"`), **independent of `imageFormat`**. When set, it overrides the extension derived from `imageFormat`; otherwise `imageFormat` applies. This lets you switch the atlas alone to WebP (`sprites: { file: "imgs/sprites.png", format: "webp" }`) without touching the generator steps — the `Sprite()` source images may stay PNG.
`retina`	`true`	Additionally generates `<name>@2x` from the `@2x` source images (which must sit next to the 1x sources).
`padding`	`0`	Spacing in pixels between the sprites.
`preloadRule`	`false`	Generates the `div.csspreload` rule in the first stylesheet.
`include`	(none)	Single images or directories (relative to skin output) to pack **without** a CSS rule (e.g. preload-only or JS assets).
`pruneSources`	`false`	After a successful atlas build, **delete source PNGs/WebPs** that now live only in the atlas (1x and `@2x`). Replaces legacy `gulp-mu-spritereducer` behaviour — **opt-in** for deploy/release builds. Never deletes the atlas itself. The build report includes `prunedSources[]`.
`pruneKeep`	`[]`	Only with `pruneSources: true`: skin-relative **files or directories** excluded from the trim. Matched against each sprite's **1x URL** (exact file or directory prefix); both `1x` and `@2x` are kept. Example: `["imgs/general/glittery", "imgs/x/y.png"]`. The build

Field	Default	Meaning
		report includes `keptSources[]`.

Resolution of the `Sprite()` source images: the path referenced in `Sprite("…")` is resolved format-independently — first the literal path, then the same name with one of the supported raster extensions (`png`, `webp`). So if a generated source image exists as PNG even though the CSS reference says `.webp` (or vice versa), the build does not abort. An error only occurs when **no** variant exists. The `@2x` variant must have the same extension as the resolved 1x image.

10.7 minify

Optional top-level field for deploy/release builds: when set, every emitted `files[].target` stylesheet is passed through a minifier on write.

Value	Effect
`false` / omitted	No minification; line breaks and indentation are preserved.
`true`	[`uglifycss`](https://www.npmjs.com/package/uglifycss) with defaults `{ maxLineLen: 1000, uglyComments: true }` (shipped with µCSS™).
object	`uglifycss` with these options merged over the defaults.
function `(css) => css`	Custom minifier (any engine), no extra dependency.

`uglifycss` is conservative: it strips whitespace and comments only — it **does not reorder or merge rules**, so the output stays semantically equivalent to the unminified CSS.

Typical deploy manifest (trim + minify):

```
minify: true,
sprites: {
    file: "imgs/sprites.png",
    retina: true,
    pruneSources: true,
    pruneKeep: ["imgs/general/glittery"]
}
```

10.8 sounds

Optional block for the **µAU** bridge — builds a sound atlas before CSS compilation (see chapter *microAU*):

Field	Meaning
`src`	Source directory (recursive `*.wav`/`*.mp3`), relative to project root
`sounds`	Alternative: explicit file list (may be combined with `src`)
`dataFile`	Output audio blob, relative to skin directory
`jsonFile`	Output JSON timing map, relative to skin directory

Field	Meaning
`format`, `mp3KBitRate` / `bitRate`, `sampleRate`, …	Passed through to `SoundAtlasMaker`. `format`: `"wav"` | `"mp3"` | `"weba"` (audio-only WebM/Opus; default from `dataFile` extension)
`triggers`	*(planned)* path to `soundTriggers.mjs` — **build time only:** extend `bindings[]` (Node, not the browser)
`handlers`	*(planned)* path to `soundHandlers.mjs` — **runtime only:** wrap/replace default handlers (browser, JSON unchanged)
`force`	Force rebuild

The build report includes `sounds[]` with `skipped`, `sounds` (names) and paths. Without a `sounds` block, copy sound files into the skin via `copyFolder`.

10.9 font

Optional block for the **µFT** bridge — builds an icon font from SVG glyphs before CSS compilation (see chapter *microFT*). Symmetric to `sounds` and `sprites`: **directory and/or individual files** register glyphs in the font even without a CSS reference.

Status: The manifest bridge is **planned** (not yet implemented in `BuildSkin`). Typical today: `FontGenerator.Create()` as a separate Gulp step, then `copyFolder` (see below).

```
font: {
    fontName: "AppSymbol",
    src: "dev/media/svg",              // recursive SVG tree, relative to project root
    include: ["dev/media/svg/extra/special-U0xE950.svg"],  // *(planned)* extra single SVGs
    outputDir: "fonts",                  // relative to skin output directory
    fontUrlBase: "../fonts/",            // URL prefix in generated CSS
    groups: {
        general: { label: "General", description: "General UI controls.", order: 1 }
    },
    glyphs: {
        "general-control-edit": { description: "Button/icon to start editing." }
    }
}
```

Multiple fonts: `font: [ { fontName: "AppSymbol", … }, { fontName: "MedicalSymbol", … } ]` — like `sounds` as an array.

Field	Meaning
`fontName`	Font family name (required) — base for file names (`AppSymbol.woff2`, …)
`src`	Source directory of SVG glyphs (recursive) **or** a TTF/OTF/WOFF/WOFF2 color/emoji font file, relative to project root
`include`	*(planned)* additional single SVG files (project-relative), combinable with `src`
`outputDir`	Target directory in the skin (font files, CSS, JSON, HTML overview)
`formats`	Extensions to emit (default: svg, ttf, eot, woff, woff2)

Field	Meaning
`fontHeight`, `normalize`, `centerHorizontally`, `classPrefix`, `fontUrlBase`	Passed through to `FontGenerator` (defaults as in µFT)
`css`, `json`, `html`	Control output or `false` to skip
`groups`, `glyphs`	Metadata for the HTML overview (**English only**)
`force`	Force rebuild

Glyph naming: `<name>-U0x<HEX>.svg` — codepoint from the file name, group id from the directory under `src` (details in chapter *microFT*).

Planned — single CSS reference: directive `-µ: Glyph("icons/edit.svg")` registers an SVG in the font and rewrites the rule with `content`/`font-family` (counterpart to `sprites.include` / `Sound()` — see `gulp-mu-au/docs/CONCEPT.md` §1).

Today without manifest bridge:

```
// 1. Gulp task (before BuildSkin): FontGenerator.Create({ fontName, src, outputDir:
"dev/media/final/fonts" })
media: [
    { copyFolder: "dev/media/final/fonts", to: "fonts", filter: "\\.(woff2?|ttf|css)$" }
]
```

Reference icon fonts in `.µ.css` via `@font-face` and classes; codepoints often as escapes in `vars` (e.g. `icon_pencil: "\\e91c"`).

10.10 files

List of stylesheets to compile — the replacement for `µ.DependentCSSFile`. `source` is relative to the source directory, `target` relative to the skin directory; both fields are required:

```
files: [
    { source: "src.µ.css", target: "std.css" },
    { source: "src_tinymce.µ.css", target: "std_tinymce.css" }
]
```

All files of a skin share variables, helpers and the sprite atlas. The preload rule goes into the first stylesheet.

11 µ evaluation context (reference)

Every `µ(...)` expression and every `-µ:` directive is evaluated in a scope that contains the following bindings. It corresponds to the old global `µ` object — just without the prefix.

11.1 The $ object

`$` contains the manifest's `vars`. Reading and writing are possible; changes apply for the rest of compilation (document order, across all `files` of a skin).

```
div.box { z-index: µ($.PanelZIndex + 1); }
```

11.2 Color and value functions

Function	Description
`Lighten(color, step, model = "hsl")`	Lightens a color relatively (positive `step`) or darkens it (negative `step`): `L' = clamp(L + L · step)`. The model `"hsl"` is bit-identical to the old µCSS™; `"oklch"` scales perceptually uniformly. Alpha is preserved.
`Alpha(color, alpha)`	Replaces a color's alpha channel. `alpha` follows the rules from "Working with colors".
`MixColors(color1, color2)`	Per-channel average of two colors (including alpha).
`AlphaValue(alpha)`	Converts an alpha specification to a byte (0–255).
`ParseColor(color)`	Parses a CSS color into the internal 32-bit representation `0xAARRGGBB`.
`FormatColor(color, alphaDecimals = 3)`	Serializes the internal representation back to CSS (`#rrggbb` or `rgba(...)`).
`PxUnit(value)`	Numbers become `"<n>px"`, everything else (e.g. `calc()` strings) is passed through unchanged.

11.3 Rule and document methods

Inside directives (and helper functions with a `this` binding) the manipulation methods of the surrounding rule are additionally available:

Method / property	Description
`AddProperty(name, value, important?)`	Appends a declaration to the rule (duplicates allowed — for fallback chains like multiple `background-image`).
`ChangeProperty(name, value, important?)`	Changes all declarations of the property, or creates one if it is missing.
`RemoveProperty(name)`	Removes all declarations of the property.
`AddRule(selector)`	Appends a new rule at the end of the document; returns the rule (with the same methods).
`InsertRule(selector)`	Inserts a new rule directly after the current rule; consecutive calls keep their order. Replacement

Method / property	Description
	for the old `AddBlock(n, µ.elementNo)`.
`rule`	The surrounding rule as a `CssRule` object (`rule.selector`, `rule.GetProperty(...)`, …).
`document`	The entire stylesheet as a `CssDocument` — e.g. for path addressing: `document.FindRule("@keyframes glittery", "from")`. Replacement for the old RememberBlocks.

11.4 Sprite()

Registers the rule's image for the sprite atlas (directive only):

```
-µ: Sprite(url, options?);
```

Parameter / option	Default	Description
`url`	— (required)	Image URL relative to the skin directory.
`offsetWidth`	`0`	Added to the computed `width`.
`offsetHeight`	`0`	Added to the computed `height`.
`offsetPosX`	`0`	Added to the `background-position` X coordinate.
`offsetPosY`	`0`	Added to the `background-position` Y coordinate.
`afterWork`	—	Hook function, runs after atlas resolution (see below).

During atlas resolution the rule's `background-image` (`url(...)` plus `image-set(...)` with retina), `background-repeat`, `background-position`, `width` and `height` are set; existing declarations of these properties are replaced.

11.5 Cursor()

Applies a cursor definition from the manifest — as a directive (`-µ: Cursor("zoom");`, rewrites the rule's `cursor` declarations) or as a value form (`cursor: µ(Cursor("zoom"));`, returns only the `url(...) x y, fallback` string). Unknown names are passed through unchanged — standard cursors like `pointer` thus work without a definition.

11.6 DataUri()

Embeds a skin-relative image file as a `data:` URI in the compiled CSS — **not** in the sprite atlas. Usable as a directive (`-µ: DataUri("imgs/icon.png");`, writes `background-image` by default) or as a value (`background-image: µ(DataUri("imgs/icon.png"));`). See the chapter [Inlining images (DataUri)](#inlining-images-datauri) for examples, options (`retina`, `wrap`, `mime`, `property`) and when to prefer `Sprite()`.

11.7 Helper functions and the this binding

Helpers from the manifest declared with `function` are called with `this` = the evaluation scope. This lets you write macros in the style of the old `µ.$.functions` — just as regular, testable JavaScript:

```
// helpers.mjs
import { Lighten } from "gulp-mu-css";

export function Borders(_baseColor, _pixelWidth, _topLighten, _rightLighten,
_bottomLighten, _leftLighten) {
    this.AddProperty("border-top", `${_pixelWidth}px solid ${Lighten(_baseColor,
_topLighten)}`);
    this.AddProperty("border-right", `${_pixelWidth}px solid ${Lighten(_baseColor,
_rightLighten)}`);
    this.AddProperty("border-bottom", `${_pixelWidth}px solid ${Lighten(_baseColor,
_bottomLighten)}`);
    this.AddProperty("border-left", `${_pixelWidth}px solid ${Lighten(_baseColor,
_leftLighten)}`);
}
```

Through `this` all scope bindings are reachable: `this.AddProperty(...)`, `this.InsertRule(...)`, `this.rule`, `this.document` and `this.$`. The binding is also kept when a helper is passed as an `afterWork` value to `Sprite()`. Arrow functions have no `this` of their own and are therefore only suitable for helpers without rule access.

Ported reference macros (`Borders`, `TableBackgrounds`, `GlitterySprite`, `FlyEx`, `FlyExUtils`) live in the µCSS™ repository under `test/fixtures/reference-macros.mjs` (demo/test fixture, not part of the npm API).

11.8 afterWork hooks

The `afterWork` hook of a sprite registration runs after the atlas has been packed and the rule rewritten. It receives a context with all result data:

Field	Description
`rule`	The rewritten rule (`CssRule`).
`document`	The stylesheet (`CssDocument`).
`url`	The registered image URL.
`baseDir`	Skin directory (for resolving neighboring files, e.g. `.json` maps).
`sprite`	`{ x, y, width, height }` — position and size in the atlas.
`atlas`	`{ file, retinaFile, width, height }` — the atlas files and total size.

Typical use: generating animation keyframes from the frame data of a `sequenceStrip` (see `GlitterySprite` in the reference macros).

12 Working with colors

12.1 Defining colors

The internal representation of a color is an unsigned 32-bit integer of the form `0xAARRGGBB` (alpha, red, green, blue; 8 bits each). All color functions accept the following input forms:

- **32-bit integer** like the internal representation, e.g. `0xffff0000` for opaque red.
- **`#` notation** with two hex digits per channel, e.g. `"#00ff00"` (alpha is set to opaque).
- **Short `#` notation** with one hex digit per channel, e.g. `"#0f0"`.
- **Extended `#` notation** with an alpha channel as the fourth component, e.g. `"#0000ff80"` for semi-transparent blue.
- **`rgb()` notation**, absolute or percentage, e.g. `"rgb(255,0,0)"` or `"rgb(100%,0%,0%)"`.
- **`rgba()` notation** with float alpha, e.g. `"rgba(255,0,0,0.5)"`.
- **CSS color names** (W3C CSS Color Module, extended list), e.g. `"red"`, `"teal"`, `"rebeccapurple"` as well as `"transparent"`.

Values outside the valid range are clipped.

12.2 Alpha values

Alpha specifications (e.g. for `Alpha(color, a)`) are possible in several forms:

- **Float** between `0.0` (transparent) and `1.0` (opaque).
- **Integer** between `0` and `255`.
- **Hex string**, e.g. `"0x80"`.
- **Percent string** from `"0%"` to `"100%"`.
- **Keywords** `"transparent"` (0), `"translucent"` (128) and `"opaque"` (255).

12.3 Color computations

The functions `Lighten`, `Alpha` and `MixColors` (chapter "µ evaluation context") cover the typical computations. Opaque results are emitted as `#rrggbb`, transparent ones as `rgba(r,g,b,a)` with three alpha decimals — identical to the old µCSS™.

```
div.menu {
    background-color: µ(Alpha($.baseBrgdColor, 0.9));
    border-top: 1px solid µ(Lighten($.baseBrgdColor, 0.3));
    border-bottom: 1px solid µ(Lighten($.baseBrgdColor, -0.3));
}
```

For new skins, the `"oklch"` model of `Lighten` is recommended: it changes the perceived lightness uniformly across all hues, whereas the legacy `"hsl"` model can jump visibly for saturated colors.

13 Node API

Besides the manifest workflow, every layer of µCSS™ is also usable directly as a Node API — for example for your own Gulp transforms or tests.

Export	Description
`CompileMcss(source, options)`	Compiles `.µ.css` source text into a `CssDocument`. Options: `vars`, `helpers`, `from` (file name for error messages), `context` (shared `MuContext`), `sprites` (SpriteManager), `cursors` (CursorManager).
`CssDocument` / `CssRule`	JSON-capable wrapper around the PostCSS AST: `FromFile`/`FromString`, `FindRule(...path)`, `FindRules(selectorOrRegex)`, `AddRule`, `GetProperty`/`AddProperty`/`ChangeProperty`/`RemoveProperty`, `ToCss()`, `ToFile()`, `ToJson()`/`FromJson()`. For special cases the raw PostCSS AST is accessible via `document.root`.
`SpriteManager`	Sprite registration and atlas resolution: `new SpriteManager({ baseDir, atlasFile, retina, padding, preloadRule, preload, writeMapFile, pruneSources, pruneKeep })`, `Register(rule, url, options)`, `await Resolve(document)` (sets `lastPruned` with `pruned[]`/`kept[]`).
`CursorManager`	Cursor definitions: `new CursorManager(definitions, { baseDir, preload })`, `Apply(rule, name)`, `Value(name)`.
`PreloadRegistry`	Collects image URLs and generates the `div.csspreload` rule.
`DefineSkin(config)` / `BuildSkin(manifest, options)`	Manifest declaration and skin build (see the chapter "Build process").
`MuContext`	Evaluation context for your own pipelines: `new MuContext({ vars, helpers })`, `Evaluate(sourceText, extraScope)`.
`Lighten`, `Alpha`, `AlphaValue`, `MixColors`, `ParseColor`, `FormatColor`, `PxUnit`	The color and value functions as direct imports.
`GetReleases`, `GetCurrentVersionString`, `GetCurrentFullVersionString`, `GetVersionHistoryString`, `StripI18xMarkup`	Version history API backed by `RELEASES.json` (see below).

Example of a JSON-based Gulp manipulation:

```
import { CssDocument } from "gulp-mu-css";

const doc = await CssDocument.FromFile("skins/src/src.µ.css");
doc.FindRules(/^div\.panel/).forEach((_rule) => _rule.AddProperty("outline", "none"));
doc.FindRule("@keyframes glittery", "from").ChangeProperty("background-position-x",
"-128px");
const json = doc.ToJson();
await doc.ToFile("out/std.css");
```

13.1 Version history API (RELEASES.json)

The package root contains a machine-readable version history `RELEASES.json` — an array of release entries, newest first, each with `main`, `minor`, `revision`, `date`, `beta` and an `info` string array (a convention originating from the Oxyd projects). The info strings carry i18x context markup (`<context="release info"/>`) so consuming applications can register them for automatic translation (e.g. via a `String.prototype.i18xRegister` i18x runtime); for plain console output the markup is stripped.

Export	Description
`GetReleases()`	Parsed `RELEASES.json`, newest first (cached).
`GetCurrentVersionString()`	`"V2.5.8"` (with β suffix for beta releases).
`GetCurrentFullVersionString()`	`"V2.5.8 · 2026-07-02 15:00"`.
`GetVersionHistoryString(options?)`	Formatted history as a string for console output. Options: `limit` (number of releases, default all), `stripMarkup` (default `true`), `indent` (default two spaces).
`StripI18xMarkup(text)`	Removes i18x tags from a single info string.

```
import { GetCurrentFullVersionString, GetVersionHistoryString } from "gulp-mu-css";

console.log(GetCurrentFullVersionString());
console.log(GetVersionHistoryString({ limit: 3 }));
// V2.5.8 · 2026-07-02 15:00
//   - DataUri(): embed skin-relative images as data: URIs in compiled CSS ...
//   - RELEASES.json version history ... with console API ...
// V2.5.7 · 2026-06-21 15:50
//   - Brand name µCSS carries the trademark sign in all documentation ...
// ...
```

14 Error diagnostics

Build errors always name the culprit together with its source location:

- **Inline JavaScript** (`µ(...)` interpolations and `-µ:` directives): errors are reported as PostCSS errors with file, line, column and a source excerpt. With multiple `µ(...)` in one value, the failing expression is named. JavaScript syntax errors appear as `invalid JavaScript expression "<source>" (...)`.

```
CssSyntaxError: skins/src/std.µ.css:2:2: microCSS: µ(NopeFn(1)): NopeFn is not defined

  1 | div.b {
> 2 |     border: 1px solid µ(NopeFn(1));
    |     ^
  3 | }
```

- **Missing sprite images**: before the atlas is packed, the existence of all source images (including `@2x` with `retina: true`) is checked. All missing files are reported together in one error — each with URL, resolved path and the referencing rule:

```
SpriteManager: 2 sprite image(s) not found:
  - "imgs/nope.png" -> C:\...\skins\std\imgs\nope.png - selector "div.bad1" (skin.µ.css:2)
  - "imgs/alsonope.png" -> C:\...\skins\std\imgs\alsonope.png - selector "div.bad2"
(skin.µ.css:3)
```

- ~~`afterWork`~~ **hooks**: errors in the hook are wrapped with the sprite URL and the rule's source position; the original error is preserved as `cause`.
- **Missing cursor images** produce only a warning (once per cursor), since the CSS stays functional via the fallback cursor.
- **media steps**: errors name the step number and type, e.g. `BuildSkin: media step 3 of 7 (buttonsAndIcons: "dev/media/buttons.psd") failed: ...`. Missing sources are checked before execution: `copy`/`copyFolder` and generator steps report the resolved path instead of a raw file system error — for `copyFolder` with the hint that the generating step (e.g. sequence-image generation into `dev/media/final/...`) probably did not run.
- **files entries**: missing source files are reported with the entry and the resolved path before compilation.

15 Migrating from µCSS™

For existing projects there is a converter tool (`tools/convert-mucss.mjs` in the µCSS™ repository) that mechanically translates the old syntax:

Old (µCSS™ 1)	New (µCSS™ 2)
`-µcss: µ.Cursor("wait");` plus `cursor: wait;`	`cursor: µ(Cursor("wait"));`
`-µcss: µ.SetBackgroundColor(µ.$.x);` plus placeholder	`background-color: µ($.x);`
`-µcss: µ.AddProperty("border", "1px solid " + µ.Lighten(...));`	`border: 1px solid µ(Lighten(...));`
`-µcss: µ.Sprite("p.png");`	`-µ: Sprite("p.png");`
`-µcss: µ.SetRememberBlock("n");`	gone — path addressing: `document.FindRule("@keyframes x", "from")`
`-µcss: //µ.X(...)` (disabled)	`/* -µ: X(...) */`
`µ.$.name = value` (in `µ.std.css`)	`vars` entry in the manifest
`µ.DefCursor(...)`	`cursors` entry in the manifest
`µ.plugins.*` / FileCopy	`media` entries in the manifest
`µ.$.Fn = function(...)«…»`	exported function in `helpers.mjs`

The `«»¡` function bodies are translated back to regular JavaScript and land as a starting point in `helpers.mjs`; manual rework is expected here.

raw → final automatically: the old µCSS™ only copied the already finished `final` folders into the skin (e.g. `flyex`, `glittery`); the strips themselves were created elsewhere (SpriteTools). The converter recognizes such `CopyFolder2Skin` calls on `dev/media/final/<…>/<name>` and — if a matching source `dev/media/raw/<name>/imgs` exists — automatically prepends the appropriate `sequenceStrip` step with `outputBase: "project"`: a **numbered PNG frame sequence** (e.g. `glittery`) becomes a folder strip, **individually named images** (e.g. the DSD images `flyex.png`/`flyexutils.png`) each become a DSD strip. This way the new pipeline rebuilds `final` reproducibly from `raw` and the following `copyFolder` takes the result into the skin.

The converter was validated against a complete legacy µCSS™ 1 code base: a comparison tool (`tools/compare-legacy-skin.mjs`, repository only) builds the converted skin and compares it rule by rule and property by property against the old compiled output — result: 2,951 rules, 0 unexpected differences (53 documented drift cases, e.g. sources edited after the last µCSS™ run).

Deliberately **not** carried over from the old µCSS™:

- **Vendor prefixes** (`-webkit-`/`-moz-`/`-ms-` duplicates): all used features have been available unprefixed for years. Vendor-specific selectors without a standard counterpart (e.g. `::-webkit-scrollbar`) are of course passed through unchanged.
- **The Photoshop dialog window** in the build: interactive parameters belong in the manifest or in environment variables.
- **FTP synchronization**: there are established tools for that today (CI/CD, rsync, gulp plugins).
- **The substitute characters «»¡**: see the chapter "Core ideas".

16 Version history

Date	Version	Notes
2013	1.0	Original µCSS™ as an Adobe Photoshop script: `-µcss:` directives, sprite atlas, PSD plugins, control via `µ.std.css`.
2026-06	2.0.0	Complete Node.js reimplementation (npm package `gulp-mu-css`, without Adobe dependency): core pipeline (M1), sprites & cursors (M2), manifest & build with incremental cache (M3), hooks & macros (M4), legacy skin migration with converter and acceptance test (M5), manual (M6).
2026-06	2.1.0	Atlas format independent of the global `imageFormat` via `sprites.format`; format-independent resolution of the `Sprite()` source images (`png`/`webp`); warning when a `copyFolder` `filter` excludes the active image format.
2026-06	2.2.0	Normalization of legacy gradient directions at compile time: `linear-gradient(top|bottom|left|right, …)` (invalid without a prefix) is raised to the standard `to …` form.
2026-06	2.2.1	Manual update (version history extended, cover-page version corrected); no functional changes compared to 2.2.0.
2026-06	2.2.2	Bilingual manual (`microCSS-de.pdf` / `microCSS-en.pdf`) and bilingual READMEs (English first) for `gulp-mu-css` and `gulp-mu-ps`; build tooling extended for multilingual manuals. No functional code changes.
2026-06	2.2.3	Fix: headings carry explicit outline levels so the auto-updated table of contents of the PDF manuals (DE/EN) is populated (was empty before). Tooling/docs fix only.

Date	Version	Notes
2026-06	2.2.4	Sound atlas integration (`sounds` block in the manifest via µAU); `sprites.include` to add standalone images or directories to the sprite atlas without a CSS rule.
2026-06	2.3.0	Build-time `@import` bundling (glob, recursive, Vue co-location); opt-in rule merge (`merge`, `@µ-override`, `onConflict`); namespace mode (`@µ-namespace`, `:global()`); build-type/variant filter (`buildFilter`, via `gulp-mu-build-filter`). Manual: Vue co-location chapter. Repository-only migration aids: `tools/convert-vue.mjs`, `tools/convert-less.mjs`. README/package metadata: GitHub monorepo links. Runtime dependencies added: `postcss-selector-parser`, `gulp-mu-build-filter`.
2026-06	2.4.x	Manual *microPS*: layer-effect tables (incl. new **stroke**/**satin**), blend modes, style transfer vs. layer links; reference PSD extended with `stroke*`/`satin*` layouts. **µPS:** `stroke` and `satin` in `Effects.mjs` with Adobe-PNG regression (`ReferenceRender.test.mjs`).
2026-06	2.4.2	Manual *microAU*/*microFT*: sound pipeline (`triggers`/`handlers`), manifest section `font`; updated PDFs (DE/EN). **µPS** 1.3.0 (stroke/satin), **µAU** 0.1.3 (sound concept docs).
2026-06	2.5.0	Manifest `minify` (CSS minification via `uglifycss`, optional custom function); `sprites.pruneKeep` protects sources from `pruneSources`; build report `keptSources[]`/`minified`. New runtime dependency

Date	Version	Notes
		`uglifycss`.
2026-06	2.5.1	Manual/PDFs (DE/EN): chapter `minify`, `sprites.pruneKeep`, build report; version history; no functional changes compared to 2.5.0.
2026-06	2.5.3	`pruneSources` also deletes sequence-strip sidecar JSON (`<strip>.json` next to a packed 1x source); protected sources keep their sidecar via `pruneKeep`.
2026-07	2.5.8	**`DataUri()`** — embed skin-relative images as `url(data:…; base64,…)` in compiled CSS (directive + value form, optional `{ retina: true }`); not packed into the sprite atlas. API: `CreateDataUriCSSValue`, `ApplyDataUri`. **`RELEASES.json`** version history (i18x-ready info strings) with console API `GetReleases`, `GetCurrentVersionString`, `GetCurrentFullVersionString`, `GetVersionHistoryString`.
2026-09	2.5.18	Manual *microFT*: OpenType color fonts and emoji fonts (COLR/CPAL v0/v1, SVG-in-OpenType, CBDT/CBLC, sbix). **µFT** 0.2.0 packages existing TTF/OTF/WOFF/WOFF2 files without dropping color tables.

17 Legal

This software is released under the MIT license and is free for both free and commercial use. Neither Dongleware nor the author is liable for any damage arising from the use of this software. Use is at your own risk; back up your work before using the tools.

Author: Meinolf Amekudzi.

17.1 MIT license (original text)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

17.2 Trademarks

Photoshop is a registered trademark of Adobe Inc.; Affinity Photo is a trademark of Serif (Europe) Ltd. Photopea is an independent online photo editor ([photopea.com](https://www.photopea.com/)). Names and products are mentioned for information only and do not constitute any misuse of the respective trade names or trademarks.

17.3 Third-party libraries

µCSS™ and µPS use third-party open-source libraries, in particular PostCSS (CSS parser, MIT license), sharp (image processing, Apache 2.0 license), ag-psd (PSD reader, MIT license) and uglifycss (CSS minification, MIT license). The sprite atlas bin packer is based on node-bin-packing (©2011 Jake Gordon and contributors, MIT license).